Delft3D FM – breakwater-intervention Mekong Delta

Stucture 1 Setup → 2 Geometry/Boundaries/Grid → 3 NE-monsoon → 4 SW-monsoon → 5 NE vs SW → 6 Storm → 7 Transect Healthy Unhealthy Mangroves & diagnostic

Important design choices:

- Every setting is in cell 1.
- Paired timing is coupled at the value of time and not on index
- the first 48 hrs are considered spin up time and excluded from the statistics.
- Maps use real mesh-polygons for flow and structured grid for waves.

1. Setup

In [1]:
```
from pathlib import Path
import time
import warnings

import numpy as np
import xarray as xr
import matplotlib
matplotlib.use('module://matplotlib_inline.backend_inline')
import matplotlib.pyplot as plt
from matplotlib.animation import FuncAnimation
from matplotlib.collections import PolyCollection
from matplotlib.colors import Normalize
from matplotlib.lines import Line2D
from matplotlib.tri import Triangulation
from IPython.display import HTML, display
from scipy.spatial import cKDTree

plt.rcParams['animation.embed_limit'] = 200
```

```
ROOT = Path.cwd()
BOUNDARY_FILE = ROOT / 'Interventie' / 'interventie_boundary_wgs84.ldb'

# Eerste bestaande kandidaat wordt gebruikt (OLD_* = terugvalpaden).
SCENARIO_FILES = {
    'ne_intervention': [ROOT / 'Interventie' / 'NE' / 'FlowFM_map.nc'],
    'sw_intervention': [ROOT / 'Interventie' / 'SW' / 'FlowFM_map.nc'],
    'ne_baseline': [
        ROOT / 'No_breakwater' / 'NE' / 'FlowFM_map' / 'FlowFM_map.nc',
        ROOT / 'No_breakwater' / 'NE' / 'OLD_FlowFM_map' / 'FlowFM_map.nc',
    ],
    'sw_baseline': [
        ROOT / 'No_breakwater' / 'SW' / 'FlowFM_map' / 'FlowFM_map.nc',
        ROOT / 'No_breakwater' / 'SW' / 'OLD_FlowFM_map' / 'FlowFM_map.nc',
    ],
    'storm': [ROOT / 'Storm' / 'FlowFM_map.nc'],
}

WAVE_FILES = {
    'ne_baseline':     ROOT / 'No_breakwater' / 'NE' / 'wavm-Wave' / 'wavm-Wave.nc',
    'sw_baseline':     ROOT / 'No_breakwater' / 'SW' / 'wavm-Wave' / 'wavm-Wave.nc',
    'ne_intervention': ROOT / 'Interventie' / 'NE' / 'wavm-Wave_inter.nc',
    'sw_intervention': ROOT / 'Interventie' / 'SW' / 'wavm-Wave_inter.nc',
    'storm':           ROOT / 'Storm' / 'wavm-Song_test.nc',
}
```

In [2]:

```
# ---------------- Alle instellingen op één plek ----------------
# Kaartuitsneden (lon, lat). De breakwaters liggen rond 105.79-105.86E, 9.23-9.26N.
X_CENTER, Y_CENTER = 105.84, 9.24
OVERVIEW_XLIM = (X_CENTER - 0.07, X_CENTER + 0.07)
OVERVIEW_YLIM = (Y_CENTER - 0.045, Y_CENTER + 0.045)
ZOOM_XLIM = (105.82, 105.88)
ZOOM_YLIM = (9.21, 9.27)

SPINUP_H = 48                       # uren die in statistieken worden overgeslagen

# Transecten: positie langs de breakwater (0 = westeinde, 1 = oosteinde).
# LET OP: dit zijn alleen posities; mangrove-eigenschappen volgen uit het model, niet uit deze labels.
TRANSECTS = {
    0.7: ('Unhealthy mangroves (0.7)', 'red'),
```

```
    0.8: ('Healthy mangroves (0.8)', 'green'),
}
OFFSHORE_KM, LEE_KM, TRANSECT_N = 0.5, 0.35, 120   # profielen
MAP_OFFSHORE_KM, MAP_LEE_KM = 3.0, 1.0             # alleen voor tekenen in kaarten
MAX_SNAP_DEG = 0.004               # flow: verder dan dit van een face -> NaN (~440 m)
MAX_SNAP_WAVE_DEG = 0.005          # golfrooster

VELOCITY_THRESHOLD = 0.6           # [m/s] drempel 'extreme snelheid' in diagnostiek
ARROW_LEN_DEG = 0.004              # pijllengte (graden) bij referentiesnelheid in zoomkaarten
```

In [3]:

```
# ---------------- Data-toegang (met caching) ----------------
_DS_CACHE, _FACE_XY, _FACE_TREE, _WAVE_TREE = {}, {}, {}, {}
_ANNOUNCED = set()


def open_ds(path):
    path = Path(path)
    if path not in _DS_CACHE:
        _DS_CACHE[path] = xr.open_dataset(path)
    return _DS_CACHE[path]


def resolve_scenario_file(name):
    candidates = SCENARIO_FILES[name]
    for cand in candidates:
        if cand.exists():
            if name not in _ANNOUNCED:
                print(f'{name}: {cand.relative_to(ROOT)}')
                _ANNOUNCED.add(name)
            return cand
    searched = '\n'.join(str(c) for c in candidates)
    raise FileNotFoundError(f'Geen bestand voor {name}. Gezocht in:\n{searched}')


def flow(name):
    return open_ds(resolve_scenario_file(name))


def wave(name):
    path = WAVE_FILES[name]
```

```
    if not path.exists():
        raise FileNotFoundError(path)
    return open_ds(path)


def time_str(t):
    return np.datetime_as_string(t, unit='m').replace('T', ' ')


def common_times(ds_a, ds_b, start=0, stride=1):
    """Gemeenschappelijke tijdstippen (op waarde). Geeft (tijden, index_a, index_b)."""
    ta, tb = ds_a['time'].values, ds_b['time'].values
    c = np.intersect1d(ta, tb)[start::stride]
    return c, np.searchsorted(ta, c), np.searchsorted(tb, c)


def face_xy(ds):
    key = id(ds)
    if key not in _FACE_XY:
        _FACE_XY[key] = (ds['mesh2d_face_x'].values, ds['mesh2d_face_y'].values)
    return _FACE_XY[key]


def assert_same_mesh(a, b):
    xa, ya = face_xy(a)
    xb, yb = face_xy(b)
    if not (np.array_equal(xa, xb, equal_nan=True) and np.array_equal(ya, yb, equal_nan=True)):
        raise ValueError('Meshes verschillen: vergelijking per face is niet mogelijk.')


def in_box(x, y, xlim, ylim):
    return np.isfinite(x) & np.isfinite(y) & (x >= xlim[0]) & (x <= xlim[1]) & (y >= ylim[0]) & (y <= ylim[1])


def face_tree(ds):
    """KD-tree over flow-face-centra (één keer per dataset). Geeft (tree, indices_van_geldige_faces)."""
    key = id(ds)
    if key not in _FACE_TREE:
        x, y = face_xy(ds)
        ok = np.flatnonzero(np.isfinite(x) & np.isfinite(y))
```

```
        _FACE_TREE[key] = (cKDTree(np.column_stack((x[ok], y[ok]))), ok)
    return _FACE_TREE[key]
```

In [4]:

```
# ---------------- Plot-hulpfuncties ----------------
def face_polygons(ds, sel):
    """Mesh-polygonen van de geselecteerde faces. Geeft (polygonen, positie-in-sel van faces met >=3 nodes)."""
    nx_, ny_ = ds['mesh2d_node_x'].values, ds['mesh2d_node_y'].values
    fn_var = ds['mesh2d_face_nodes']
    start = int(fn_var.attrs.get('start_index', 1))
    fn = fn_var.values[sel]
    polys, keep = [], []
    for k, row in enumerate(fn):
        ids = row[np.isfinite(row)].astype(np.int64) - start
        ids = ids[(ids >= 0) & (ids < nx_.size)]
        if ids.size >= 3:
            polys.append(np.column_stack((nx_[ids], ny_[ids])))
            keep.append(k)
    return polys, np.asarray(keep, dtype=int)


def add_polys(ax, polys, values, cmap, vmin=None, vmax=None, norm=None):
    pc = PolyCollection(polys, array=np.ma.masked_invalid(np.asarray(values, float)), cmap=cmap,
                        norm=norm or Normalize(vmin, vmax), edgecolors='none', antialiased=False)
    ax.add_collection(pc)
    return pc


def structured_triangulation(x, y, active):
    """Triangulatie op de (nmax, mmax)-connectiviteit van het golfrooster.
    Cellen met een inactief/niet-eindig hoekpunt worden gemaskeerd, dus er ontstaan geen driehoeken
    over land of over de inham (geen Delaunay-artefacten, geen handmatige maskers nodig).
    Waardenarrays moeten als np.nan_to_num(values).ravel() worden doorgegeven."""
    ny, nx = x.shape
    idx = np.arange(ny * nx).reshape(ny, nx)
    a, b = idx[:-1, :-1].ravel(), idx[:-1, 1:].ravel()
    c, d = idx[1:, 1:].ravel(), idx[1:, :-1].ravel()
    tris = np.concatenate([np.column_stack((a, b, c)), np.column_stack((a, c, d))])
    ok = (active & np.isfinite(x) & np.isfinite(y)).ravel()
    bad = ~(ok[a] & ok[b] & ok[c] & ok[d])
    xf = np.where(np.isfinite(x), x, 0.0).ravel()
```

```
    yf = np.where(np.isfinite(y), y, 0.0).ravel()
    return Triangulation(xf, yf, tris, mask=np.concatenate([bad, bad]))


def make_arrow_grid(x, y, xlim, ylim, nx=30, ny=20):
    """Regelmatig vakkenraster voor pijlen. sel = indices (in x,y) van de faces binnen de uitsnede."""
    sel = np.flatnonzero(in_box(x, y, xlim, ylim))
    if sel.size < 3:
        raise ValueError('Te weinig faces in de uitsnede')
    xe, ye = np.linspace(xlim[0], xlim[1], nx + 1), np.linspace(ylim[0], ylim[1], ny + 1)
    cx = np.clip(np.digitize(x[sel], xe) - 1, 0, nx - 1)
    cy = np.clip(np.digitize(y[sel], ye) - 1, 0, ny - 1)
    cid = cy * nx + cx
    occ = np.unique(cid)
    gx, gy = np.meshgrid((xe[:-1] + xe[1:]) / 2, (ye[:-1] + ye[1:]) / 2)
    return dict(sel=sel, cid=cid, occ=occ, n=nx * ny, x=gx.ravel()[occ], y=gy.ravel()[occ])


def bin_vectors(grid, u, v):
    """Gemiddelde u,v per rastervak. u,v: (tijd, len(grid['sel'])). Lege vakken -> NaN (geen pijl)."""
    out = []
    for arr in (u, v):
        res = np.full((arr.shape[0], grid['occ'].size), np.nan)
        for k in range(arr.shape[0]):
            ok = np.isfinite(arr[k])
            cnt = np.bincount(grid['cid'][ok], minlength=grid['n'])[grid['occ']]
            tot = np.bincount(grid['cid'][ok], weights=arr[k][ok], minlength=grid['n'])[grid['occ']]
            np.divide(tot, cnt, out=res[k], where=cnt > 0)
        out.append(res)
    return out[0], out[1]


def show_anim(anim, fps=5):
    display(HTML(anim.to_jshtml(fps=fps)))
```

2. Geometry: boundaries (.ldb), breakwater, grids, and transects

In [5]:
```
def read_ldb(path):
    lines = [ln.strip() for ln in Path(path).read_text(encoding='utf-8').splitlines()
```

```
            if ln.strip() and not ln.strip().startswith('*')]
    out, cur = [], 0
    while cur < len(lines):
        name = lines[cur]
        count, dims = map(int, lines[cur + 1].split()[:2])
        if dims != 2:
            raise ValueError(f'Onverwachte dimensies in {name}')
        pts = np.array([list(map(float, lines[cur + 2 + j].split()[:2])) for j in range(count)])
        out.append((name, pts[:, 0], pts[:, 1]))
        cur += count + 2
    return out


BOUNDARIES_ALL = read_ldb(BOUNDARY_FILE)
local_boundaries = [(n, bx, by) for n, bx, by in BOUNDARIES_ALL
                    if bx.max() >= OVERVIEW_XLIM[0] and bx.min() <= OVERVIEW_XLIM[1]
                    and by.max() >= OVERVIEW_YLIM[0] and by.min() <= OVERVIEW_YLIM[1]]
print(f'Boundaries totaal: {len(BOUNDARIES_ALL)}, in kaartuitsnede: {len(local_boundaries)}')


def draw_boundaries(ax, lw=2, color='black'):
    for _, bx, by in local_boundaries:
        ax.plot(bx, by, color=color, linewidth=lw, zorder=4)
```

```
Boundaries totaal: 259, in kaartuitsnede: 27
```

In [6]:
```
# --- Breakwater-segmenten, lokale km-coördinaten en hoofdrichting ---
BW_LON, BW_LAT = (105.78, 105.87), (9.22, 9.26)
bw_segments = [(n, bx, by) for n, bx, by in BOUNDARIES_ALL
               if bx.min() >= BW_LON[0] and bx.max() <= BW_LON[1]
               and by.min() >= BW_LAT[0] and by.max() <= BW_LAT[1]]
print(f'{len(bw_segments)} breakwater-segmenten: {bw_segments[0][0]} ... {bw_segments[-1][0]}')

bw_lon = np.concatenate([s[1] for s in bw_segments])
bw_lat = np.concatenate([s[2] for s in bw_segments])

LON0, LAT0 = bw_lon.mean(), bw_lat.mean()
KX, KY = 111.32 * np.cos(np.deg2rad(LAT0)), 110.57          # km per graad
to_km = lambda lon, lat: np.column_stack(((np.asarray(lon) - LON0) * KX, (np.asarray(lat) - LAT0) * KY))
to_lonlat = lambda xy: (LON0 + xy[:, 0] / KX, LAT0 + xy[:, 1] / KY)
```

```
bw_pts = to_km(bw_lon, bw_lat)
_, _, vt = np.linalg.svd(bw_pts - bw_pts.mean(axis=0))
bw_dir = vt[0] if vt[0][0] > 0 else -vt[0]                  # langs de breakwater (oostwaarts)
bw_normal = np.array([bw_dir[1], -bw_dir[0]])               # loodrecht, wijst naar zuidoost = offshore
bw_t = bw_pts @ bw_dir
n_spine = np.median(bw_pts @ bw_normal)                     # positie van de rug (spurs tellen niet mee)
BW_LENGTH_KM = bw_t.max() - bw_t.min()
print(f'Hoek t.o.v. oost: {np.degrees(np.arctan2(bw_dir[1], bw_dir[0])):.1f} graden | lengte: {BW_LENGTH_KM:.2f} km')
print(f'Offshore-richting (oost, noord): ({bw_normal[0]:+.2f}, {bw_normal[1]:+.2f}) -> verwacht (+, -)')


def bw_crossing_point(frac):
    """Echt snijpunt van het transect met de breakwater. Valt het transect in een opening, dan wordt de rug gebruikt."""
    t_c = bw_t.min() + frac * BW_LENGTH_KM
    hits = []
    for _, bx, by in bw_segments:
        p = to_km(bx, by)
        tt = p @ bw_dir
        for k in range(len(p) - 1):
            t1, t2 = tt[k], tt[k + 1]
            if t1 != t2 and (t1 - t_c) * (t2 - t_c) <= 0:
                alpha = (t_c - t1) / (t2 - t1)
                hits.append(p[k] + alpha * (p[k + 1] - p[k]))
    if not hits:
        print(f'Waarschuwing: geen snijpunt bij fractie {frac}; rugpositie gebruikt.')
        return bw_dir * t_c + bw_normal * n_spine
    hits = np.asarray(hits)
    return hits[np.argmin(np.abs(hits @ bw_normal - n_spine))]


def transect_xy(fraction, offshore_km=None, lee_km=None, n=None):
    """s < 0 offshore, s = 0 breakwater, s > 0 lee (km). Geeft (s, xy in km)."""
    offshore_km = OFFSHORE_KM if offshore_km is None else offshore_km
    lee_km = LEE_KM if lee_km is None else lee_km
    n = TRANSECT_N if n is None else n
    crossing = bw_crossing_point(fraction)
    s = np.linspace(-offshore_km, lee_km, n)
    return s, crossing[None, :] - bw_normal[None, :] * s[:, None]

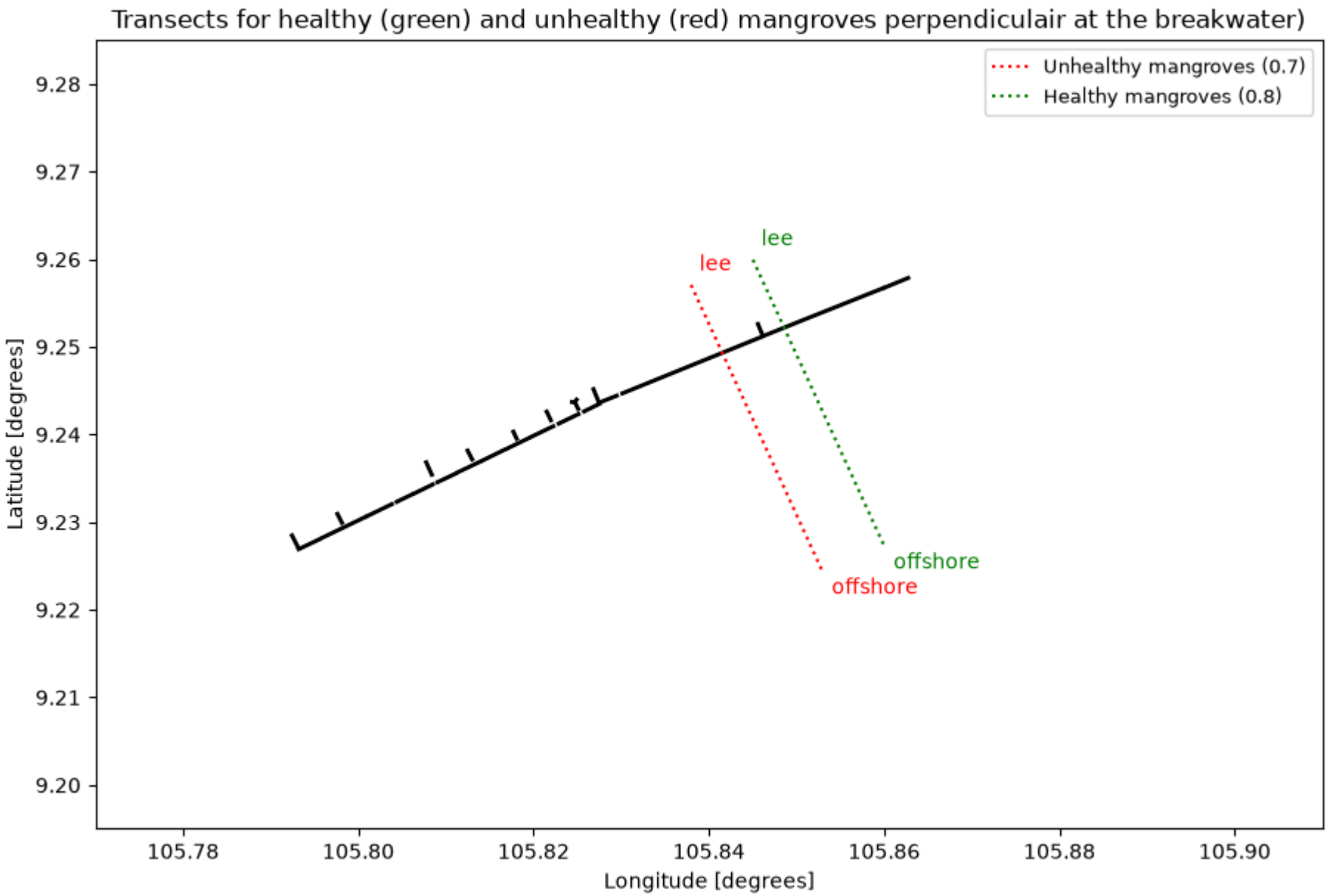

def draw_transects(ax, offshore_km=MAP_OFFSHORE_KM, lee_km=MAP_LEE_KM):
```

```
    for frac, (_, color) in TRANSECTS.items():
        _, xy = transect_xy(frac, offshore_km, lee_km)
        lo, la = to_lonlat(xy)
        ax.plot(lo, la, color=color, linestyle=':', linewidth=1.5, zorder=5)


def transect_legend():
    handles = [Line2D([0], [0], color=c, linestyle=':', linewidth=1.5, label=n) for n, c in TRANSECTS.values()]
    handles.append(Line2D([0], [0], color='black', linewidth=1.5, label='Breakwater'))
    return handles


# --- Controleplot: staan de transecten loodrecht op de breakwater? ---
fig, ax = plt.subplots(figsize=(9, 6.5), constrained_layout=True)
for _, bx, by in bw_segments:
    ax.plot(bx, by, color='black', linewidth=2)
for frac, (name, color) in TRANSECTS.items():
    _, xy = transect_xy(frac, MAP_OFFSHORE_KM, MAP_LEE_KM)
    lo, la = to_lonlat(xy)
    ax.plot(lo, la, color=color, linestyle=':', linewidth=1.5, label=name)
    ax.annotate('offshore', (lo[0], la[0]), color=color, xytext=(5, -12), textcoords='offset points')
    ax.annotate('lee', (lo[-1], la[-1]), color=color, xytext=(5, 5), textcoords='offset points')
ax.set(xlim=OVERVIEW_XLIM, ylim=OVERVIEW_YLIM, aspect='equal', xlabel='Longitude [degrees]',
       ylabel='Latitude [degrees]', title='Transects for healthy (green) and unhealthy (red) mangroves perpendiculair at the b
ax.legend(loc='upper right', fontsize=9)
plt.show()
```

```
27 breakwater-segmenten: Boundary_0020 ... Boundary_0070
Hoek t.o.v. oost: 24.5 graden | lengte: 8.38 km
Offshore-richting (oost, noord): (+0.41, -0.91) -> verwacht (+, -)
```



3. NE-monsoon (february 2023): with and without intervention

Arrow length is scaled with velocity (mean per cell); Reference arrow is the 95e-percentile-velocity. The transect are from 3 km offshore / 1 km lee; the profiles used in section 7 are from 0,5 km / 0,35 km.

In [7]:
```
def flow_comparison_animation(key_i, key_b, label, frame_step=24, grid_n=(30, 30)):
    """Animatie waterdiepte + stroomrichting: links met interventie, rechts baseline."""
    ds_i, ds_b = flow(key_i), flow(key_b)
    assert_same_mesh(ds_i, ds_b)
    x, y = face_xy(ds_i)
    grid = make_arrow_grid(x, y, ZOOM_XLIM, ZOOM_YLIM, nx=grid_n[0], ny=grid_n[1])
    sel = grid['sel']
    polys, keep = face_polygons(ds_i, sel)
    _, ii, ib = common_times(ds_i, ds_b, start=1, stride=frame_step)
    times = ds_i['time'].values[ii]

    data = {}
    for tag, ds, idx in (('i', ds_i, ii), ('b', ds_b, ib)):
        pick = dict(time=idx, mesh2d_nFaces=sel)
        u = ds['mesh2d_ucx'].isel(**pick).values
        v = ds['mesh2d_ucy'].isel(**pick).values
        d = ds['mesh2d_waterdepth'].isel(**pick).values
        ub, vb = bin_vectors(grid, u, v)
        data[tag] = dict(u=ub, v=vb, depth=np.nan_to_num(d[:, keep]), speed=np.hypot(u, v))

    ref = float(np.nanpercentile(np.r_[data['i']['speed'].ravel(), data['b']['speed'].ravel()], 95))
    fac = ARROW_LEN_DEG / max(ref, 1e-6)
    vmax = float(np.nanpercentile(np.r_[data['i']['depth'].ravel(), data['b']['depth'].ravel()], 99.5))

    fig, axes = plt.subplots(1, 2, figsize=(16, 7), constrained_layout=True)
    colls, quivs = [], []
    for ax, tag, title in zip(axes, ('i', 'b'), (f'{label} with intervention', f'{label} without intervention')):
        colls.append(add_polys(ax, polys, data[tag]['depth'][0], 'Blues', 0, vmax))
        quivs.append(ax.quiver(grid['x'], grid['y'], data[tag]['u'][0] * fac, data[tag]['v'][0] * fac,
                               color='black', angles='xy', scale_units='xy', scale=1, width=0.0015, zorder=3))
        if tag == 'i':
            draw_boundaries(ax)
        draw_transects(ax)
        ax.set(xlim=ZOOM_XLIM, ylim=ZOOM_YLIM, aspect='equal', xlabel='Longitude [degrees]', ylabel='Latitude [degrees]')
        ax.set_title(title)
    axes[0].legend(handles=transect_legend(), loc='upper right', fontsize=9, framealpha=0.9)
    quivs[1].axes.quiverkey(quivs[1], 0.85, 1.03, ref * fac, f'{ref:.2f} m/s', labelpos='E', coordinates='axes')
```

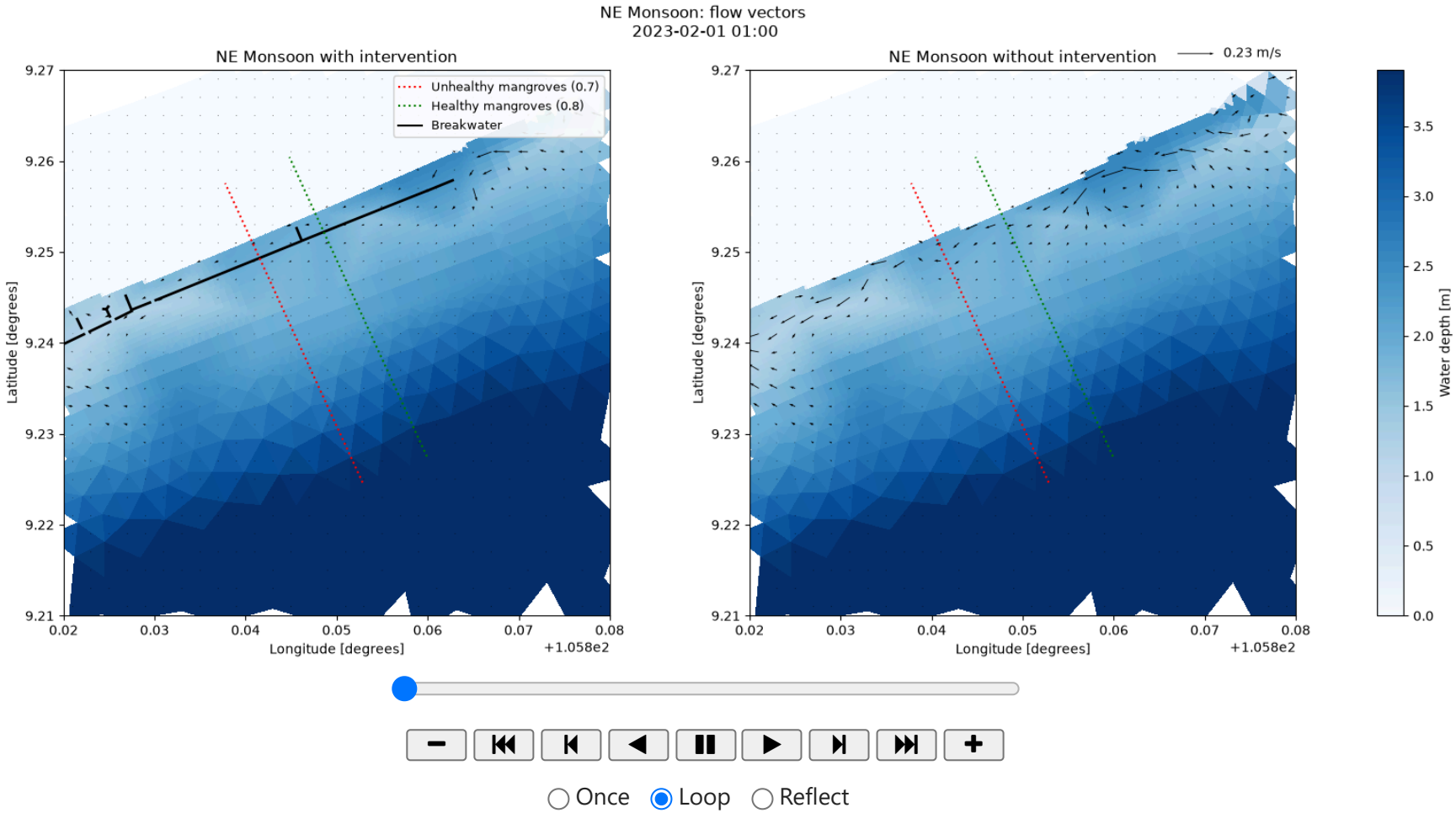
```
    fig.colorbar(colls[0], ax=axes, label='Water depth [m]')
    title = fig.suptitle(f'{label}: flow vectors \n{time_str(times[0])}')

    def update(k):
        for coll, tag in zip(colls, ('i', 'b')):
            coll.set_array(np.ma.masked_invalid(data[tag]['depth'][k]))
        for q, tag in zip(quivs, ('i', 'b')):
            q.set_UVC(data[tag]['u'][k] * fac, data[tag]['v'][k] * fac)
        title.set_text(f'{label}: flow vectors \n{time_str(times[k])}')
        return (*colls, *quivs, title)

    anim = FuncAnimation(fig, update, frames=len(times), interval=200, blit=False, repeat=True)
    plt.close(fig)
    return anim


ne_flow_anim = flow_comparison_animation('ne_intervention', 'ne_baseline', 'NE Monsoon')
show_anim(ne_flow_anim)
```

```
ne_intervention: Interventie\NE\FlowFM_map.nc
ne_baseline: No_breakwater\NE\FlowFM_map\FlowFM_map.nc
```



In [8]:
```python
def flow_difference(key_i, key_b, label, stride=6, threshold=0.01, xlim=ZOOM_XLIM, ylim=ZOOM_YLIM):
    """Verschil in snelheid (interventie - baseline): tijdgemiddeld, mediaan |verschil| en tijdreeks."""
    ds_i, ds_b = flow(key_i), flow(key_b)
    assert_same_mesh(ds_i, ds_b)
    x, y = face_xy(ds_i)
    sel = np.flatnonzero(in_box(x, y, xlim, ylim))
    polys, keep = face_polygons(ds_i, sel)
    sel = sel[keep]
    xs, ys = x[sel], y[sel]
    c, ii, ib = common_times(ds_i, ds_b, start=SPINUP_H, stride=stride)
    print(f'{label}: {sel.size} faces, {c.size} tijdstappen ({time_str(c[0])} -> {time_str(c[-1])})')
```

```
def uv(ds, idx):
    pick = dict(time=idx, mesh2d_nFaces=sel)
    return ds['mesh2d_ucx'].isel(**pick).values, ds['mesh2d_ucy'].isel(**pick).values

u_i, v_i = uv(ds_i, ii)
u_b, v_b = uv(ds_b, ib)
du, dv = u_i - u_b, v_i - v_b
dspeed = np.hypot(u_i, v_i) - np.hypot(u_b, v_b)       # negatief = interventie langzamer
abs_d = np.abs(dspeed)

with warnings.catch_warnings():
    warnings.simplefilter('ignore', RuntimeWarning)
    max_t, mean_t, med_t = np.nanmax(abs_d, axis=1), np.nanmean(abs_d, axis=1), np.nanmedian(abs_d, axis=1)
    mean_map, med_map = np.nanmean(dspeed, axis=0), np.nanmedian(abs_d, axis=0)
    mean_du, mean_dv = np.nanmean(du, axis=0), np.nanmean(dv, axis=0)
    pcts = np.nanpercentile(abs_d, [50, 95, 99, 100])

f_max = int(np.nanargmax(max_t))
p_max = int(np.nanargmax(abs_d[f_max]))
print(f'|Δsnelheid| over alle faces en tijden: P50 {pcts[0]:.3f}, P95 {pcts[1]:.3f}, P99 {pcts[2]:.3f}, max {pcts[3]:.3f}
print(f'Grootste verschil: {dspeed[f_max, p_max]:+.3f} m/s op {time_str(c[f_max])}, lon {xs[p_max]:.4f}, lat {ys[p_max]:.4

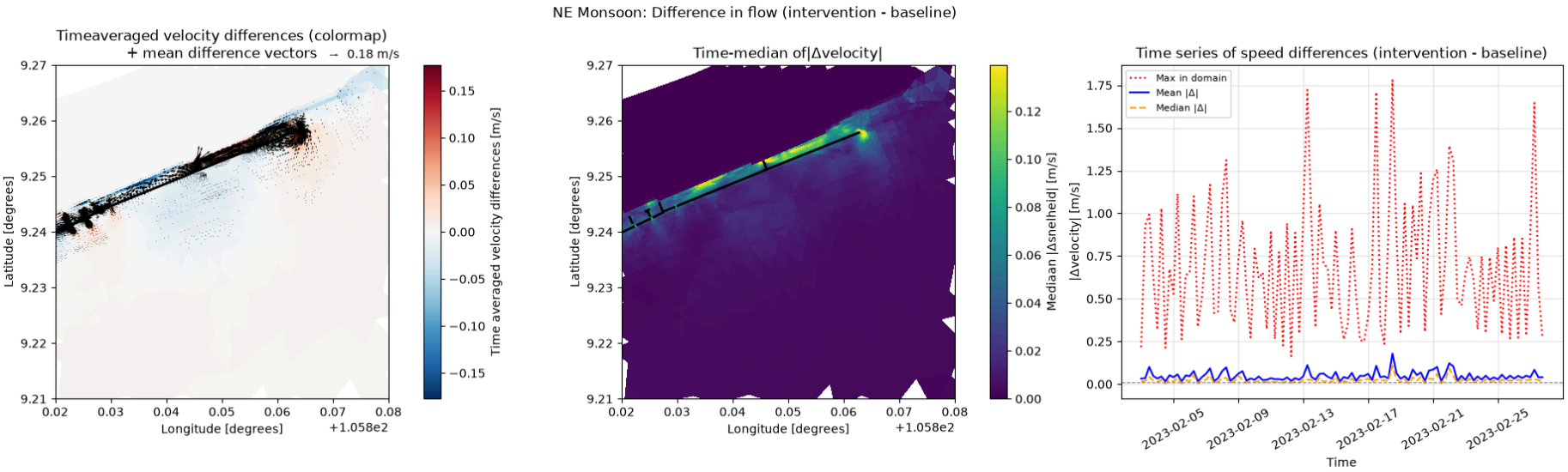
fig, axes = plt.subplots(1, 3, figsize=(19, 5.5), constrained_layout=True)
lim = max(float(np.nanpercentile(np.abs(mean_map), 98)), 1e-6)
pc1 = add_polys(axes[0], polys, mean_map, 'RdBu_r', -lim, lim)
fig.colorbar(pc1, ax=axes[0], label='Time averaged velocity differences [m/s]')
qsel = np.flatnonzero(np.hypot(mean_du, mean_dv) > threshold)[::2]
if qsel.size:
    q = axes[0].quiver(xs[qsel], ys[qsel], mean_du[qsel], mean_dv[qsel], angles='xy', color='black', width=0.002)
    axes[0].quiverkey(q, 0.85, 1.03, lim, f'{lim:.2f} m/s', labelpos='E', coordinates='axes')
axes[0].set_title('Timeaveraged velocity differences (colormap)\n+ mean difference vectors')

vmax2 = max(float(np.nanpercentile(med_map, 99)), 1e-6)
pc2 = add_polys(axes[1], polys, med_map, 'viridis', 0, vmax2)
fig.colorbar(pc2, ax=axes[1], label='Mediaan |Δsnelheid| [m/s]')
axes[1].set_title('Time-median of|Δvelocity|')
for ax in axes[:2]:
    draw_boundaries(ax)
    ax.set(xlim=xlim, ylim=ylim, aspect='equal', xlabel='Longitude [degrees]', ylabel='Latitude [degrees]')
```

```
    axes[2].plot(c, max_t, color='red', linestyle=':', label='Max in domain')
    axes[2].plot(c, mean_t, color='blue', label='Mean |Δ|')
    axes[2].plot(c, med_t, color='orange', linestyle='--', label='Median |Δ|')
    axes[2].axhline(threshold, color='grey', linestyle='--', linewidth=0.8)
    axes[2].set(xlabel='Time', ylabel='|Δvelocity| [m/s]', title='Time series of speed differences (intervention - baseline)')
    axes[2].legend(fontsize=8)
    axes[2].grid(alpha=0.3)
    axes[2].tick_params(axis='x', rotation=30)
    fig.suptitle(f'{label}: Difference in flow (intervention - baseline)')
    plt.show()


flow_difference('ne_intervention', 'ne_baseline', 'NE Monsoon')
```

```
NE Monsoon: 11651 faces, 100 tijdstappen (2023-02-03 00:00 -> 2023-02-27 18:00)
|Δsnelheid| over alle faces en tijden: P50 0.019, P95 0.186, P99 0.423, max 1.780 m/s
Grootste verschil: +1.780 m/s op 2023-02-18 12:00, lon 105.8299, lat 9.2447
```



3b. NE-monsoon: significant wave height H_s

In [9]:
```
def load_hs_block(ds, time_idx, xlim, ylim, margin=1):
    """Lees Hs voor een rechthoekig blok van het gestructureerde golfrooster. Inactieve cellen -> NaN."""
    x, y = ds['x'].values, ds['y'].values
```

```
    rows, cols = np.where(in_box(x, y, xlim, ylim))
    if rows.size == 0:
        raise ValueError('Geen golfcellen in de uitsnede')
    r = slice(max(rows.min() - margin, 0), rows.max() + margin + 1)
    c = slice(max(cols.min() - margin, 0), cols.max() + margin + 1)
    xs, ys = x[r, c], y[r, c]
    active = (ds['kcs'].isel(nmax=r, mmax=c).values > 0) & np.isfinite(xs) & np.isfinite(ys)
    hs = ds['hsign'].isel(time=time_idx, nmax=r, mmax=c).values.astype(float)
    return xs, ys, np.where(active[None], hs, np.nan), active

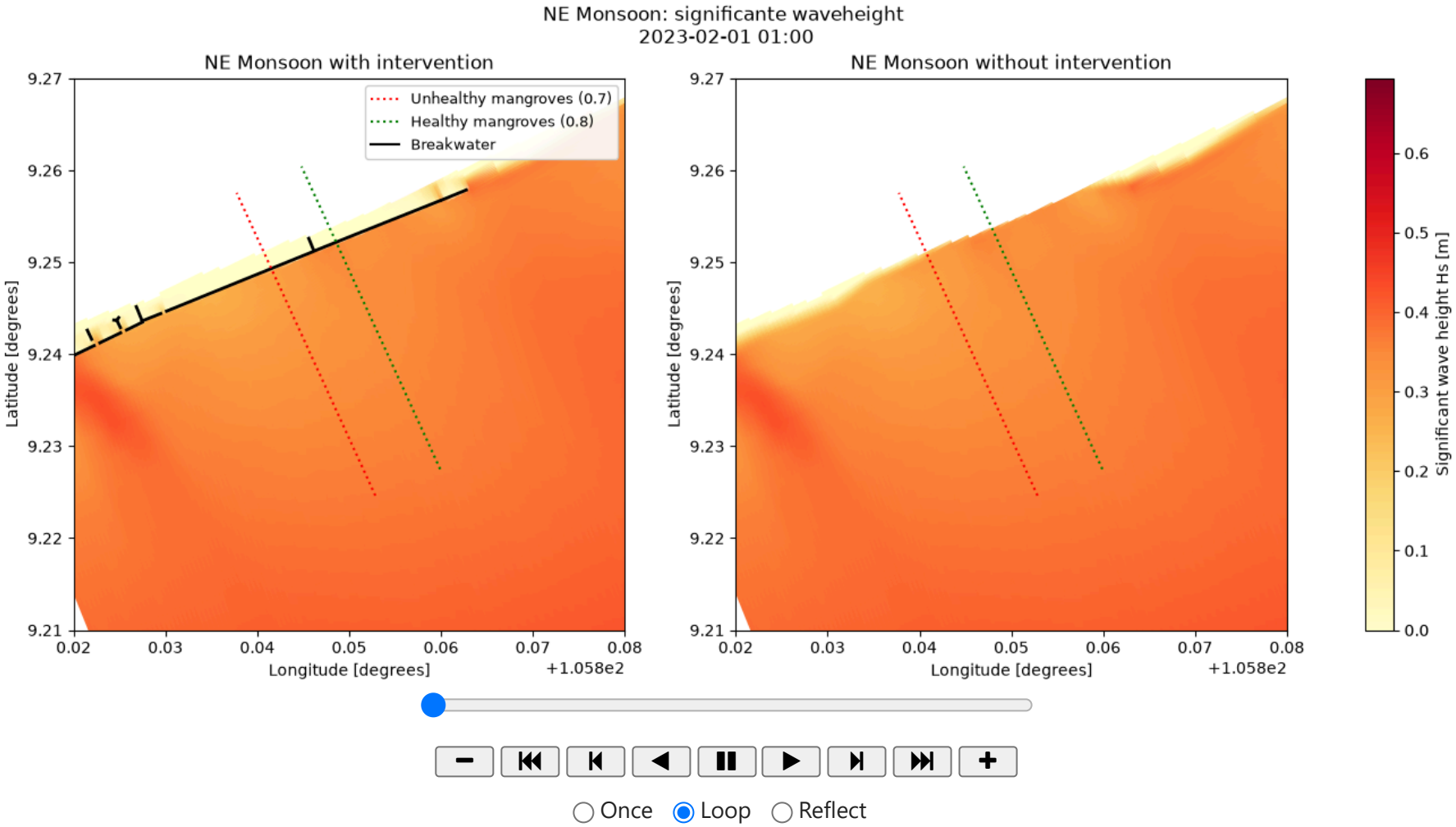

def hs_comparison_animation(key_i, key_b, label, stride=3):
    ds_i, ds_b = wave(key_i), wave(key_b)
    _, ii, ib = common_times(ds_i, ds_b, start=1, stride=stride)
    times = ds_i['time'].values[ii]
    xs, ys, hs_i, act_i = load_hs_block(ds_i, ii, ZOOM_XLIM, ZOOM_YLIM)
    xb, yb, hs_b, act_b = load_hs_block(ds_b, ib, ZOOM_XLIM, ZOOM_YLIM)
    tri_i, tri_b = structured_triangulation(xs, ys, act_i), structured_triangulation(xb, yb, act_b)
    vmax = float(max(np.nanpercentile(hs_i, 99.5), np.nanpercentile(hs_b, 99.5)))
    if not np.isfinite(vmax) or vmax <= 0:
        vmax = 1.0

    fig, axes = plt.subplots(1, 2, figsize=(13, 6), constrained_layout=True)
    meshes = []
    for ax, tri, hs, title in zip(axes, (tri_i, tri_b), (hs_i, hs_b),
                                  (f'{label} with intervention', f'{label} without intervention')):
        meshes.append(ax.tripcolor(tri, np.nan_to_num(hs[0]).ravel(), shading='gouraud', cmap='YlOrRd', vmin=0, vmax=vmax))
        draw_transects(ax)
        ax.set(xlim=ZOOM_XLIM, ylim=ZOOM_YLIM, aspect='equal', xlabel='Longitude [degrees]', ylabel='Latitude [degrees]')
        ax.set_title(title)
    draw_boundaries(axes[0])
    axes[0].legend(handles=transect_legend(), loc='upper right', fontsize=9, framealpha=0.9)
    fig.colorbar(meshes[0], ax=axes, label='Significant wave height Hs [m]')
    title = fig.suptitle(f'{label}: significante wave height \n{time_str(times[0])}')

    def update(k):
        meshes[0].set_array(np.nan_to_num(hs_i[k]).ravel())
        meshes[1].set_array(np.nan_to_num(hs_b[k]).ravel())
        title.set_text(f'{label}: significante waveheight \n{time_str(times[k])}')
        return (*meshes, title)
```

```
    anim = FuncAnimation(fig, update, frames=len(times), interval=200, blit=False, repeat=True)
    plt.close(fig)
    return anim


ne_hs_anim = hs_comparison_animation('ne_intervention', 'ne_baseline', 'NE Monsoon')
show_anim(ne_hs_anim)
```



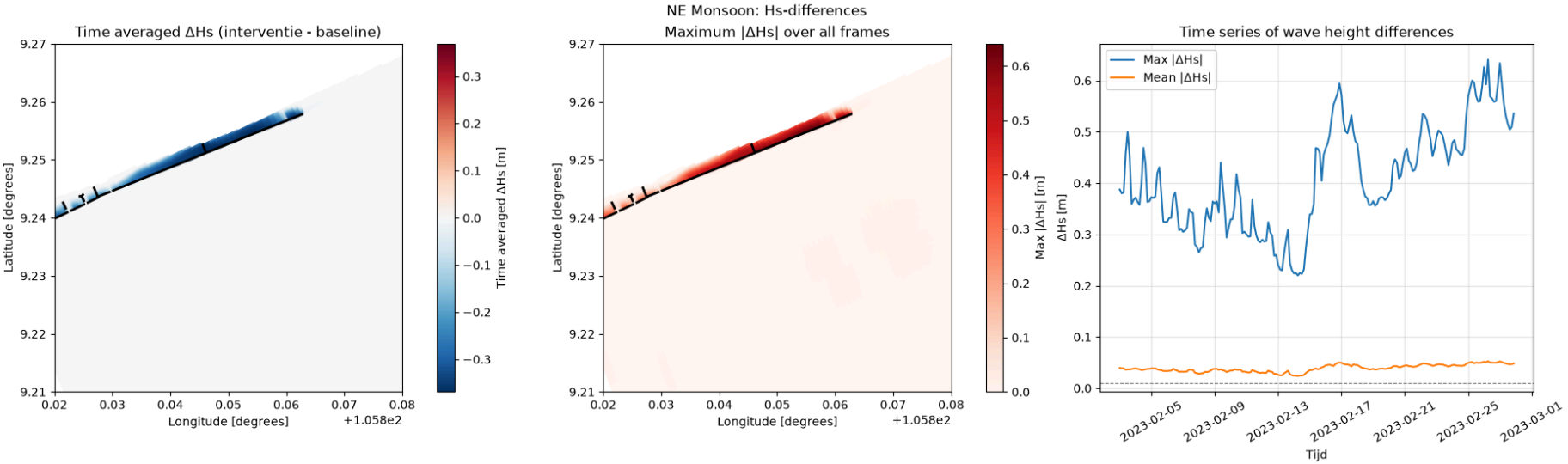
In [10]:
```
def hs_difference(key_i, key_b, label, stride=3, threshold=0.01):
    """Hs-verschil (interventie - baseline): tijdgemiddeld (met teken), maximum |verschil| en tijdreeks."""
```

```
    ds_i, ds_b = wave(key_i), wave(key_b)
    c, ii, ib = common_times(ds_i, ds_b, start=SPINUP_H, stride=stride)
    xs, ys, hs_i, act = load_hs_block(ds_i, ii, ZOOM_XLIM, ZOOM_YLIM)
    _, _, hs_b, _ = load_hs_block(ds_b, ib, ZOOM_XLIM, ZOOM_YLIM)
    diff = hs_i - hs_b                                      # negatief = interventie lager
    ad = np.abs(diff)
    with warnings.catch_warnings():
        warnings.simplefilter('ignore', RuntimeWarning)
        max_t, mean_t = np.nanmax(ad, axis=(1, 2)), np.nanmean(ad, axis=(1, 2))
        mean_map, max_map = np.nanmean(diff, axis=0), np.nanmax(ad, axis=0)
    f = int(np.nanargmax(max_t))
    p = np.unravel_index(int(np.nanargmax(ad[f])), ad[f].shape)
    n_act = int(np.isfinite(max_map).sum())
    n_diff = int(np.nansum(max_map > threshold))
    print(f'--- {label} ---  ({c.size} tijdstappen, spin-up weggelaten)')
    print(f'Grootste verschil: {diff[f][p]:+.3f} m op {time_str(c[f])}, lon {xs[p]:.4f}, lat {ys[p]:.4f}')
    print(f'Punten met |ΔHs| > {threshold} m (ooit): {n_diff} van {n_act} actieve golfcellen')
    print(f'Gemiddeld |ΔHs| in uitsnede: {np.nanmean(mean_t):.3f} m')

    tri = structured_triangulation(xs, ys, act)
    fig, axes = plt.subplots(1, 3, figsize=(19, 5.5), constrained_layout=True)
    lim = max(float(np.nanpercentile(np.abs(mean_map), 99)), 1e-6)
    m0 = axes[0].tripcolor(tri, np.nan_to_num(mean_map).ravel(), shading='gouraud', cmap='RdBu_r', vmin=-lim, vmax=lim)
    fig.colorbar(m0, ax=axes[0], label='Time averaged ΔHs [m]')
    axes[0].set_title('Time averaged ΔHs (interventie - baseline)')
    m1 = axes[1].tripcolor(tri, np.nan_to_num(max_map).ravel(), shading='gouraud', cmap='Reds',
                           vmin=0, vmax=max(float(np.nanmax(max_map)), 1e-6))
    fig.colorbar(m1, ax=axes[1], label='Max |ΔHs| [m]')
    axes[1].set_title('Maximum |ΔHs| over all frames')
    for ax in axes[:2]:
        draw_boundaries(ax)
        ax.set(xlim=ZOOM_XLIM, ylim=ZOOM_YLIM, aspect='equal', xlabel='Longitude [degrees]', ylabel='Latitude [degrees]')
    axes[2].plot(c, max_t, label='Max |ΔHs|')
    axes[2].plot(c, mean_t, label='Mean |ΔHs|')
    axes[2].axhline(threshold, color='grey', linestyle='--', linewidth=0.8)
    axes[2].set(xlabel='Tijd', ylabel='ΔHs [m]', title='Time series of wave height differences')
    axes[2].legend()
    axes[2].grid(alpha=0.3)
    axes[2].tick_params(axis='x', rotation=30)
    fig.suptitle(f'{label}: Hs-differences')
    plt.show()
```

```
hs_difference('ne_intervention', 'ne_baseline', 'NE Monsoon')
```

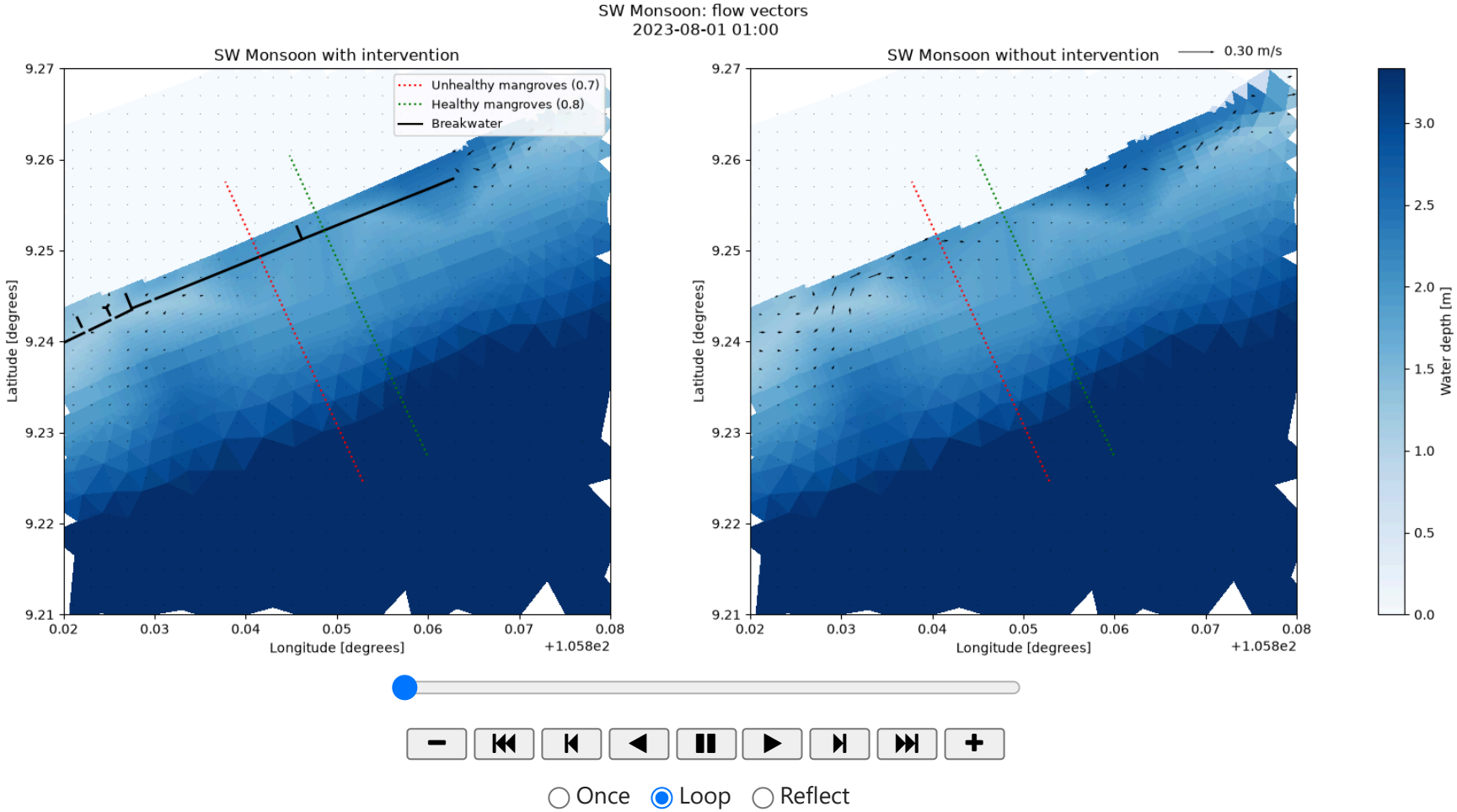
```
--- NE Monsoon ---  (200 tijdstappen, spin-up weggelaten)
Grootste verschil: -0.641 m op 2023-02-26 06:00, lon 105.8551, lat 9.2549
Punten met |ΔHs| > 0.01 m (ooit): 560 van 3189 actieve golfcellen
Gemiddeld |ΔHs| in uitsnede: 0.039 m
```



4. SW-monsoon (augusst 2023): with and without intervention

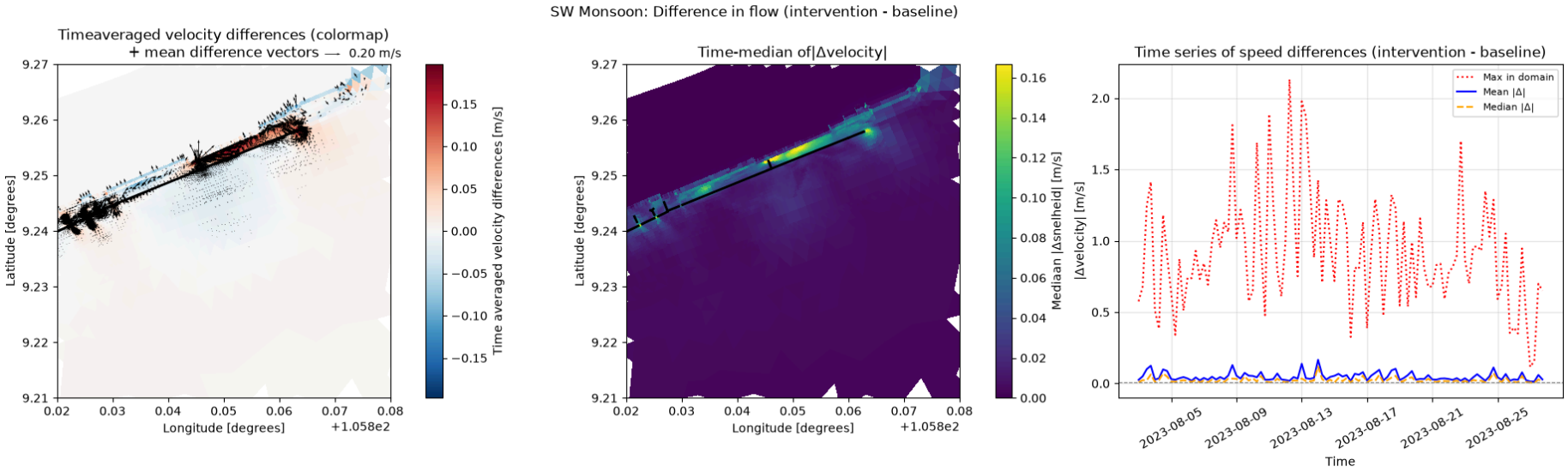
In [11]:
```
sw_flow_anim = flow_comparison_animation('sw_intervention', 'sw_baseline', 'SW Monsoon')
show_anim(sw_flow_anim)
```

```
sw_intervention: Interventie\SW\FlowFM_map.nc
sw_baseline: No_breakwater\SW\FlowFM_map\FlowFM_map.nc
```

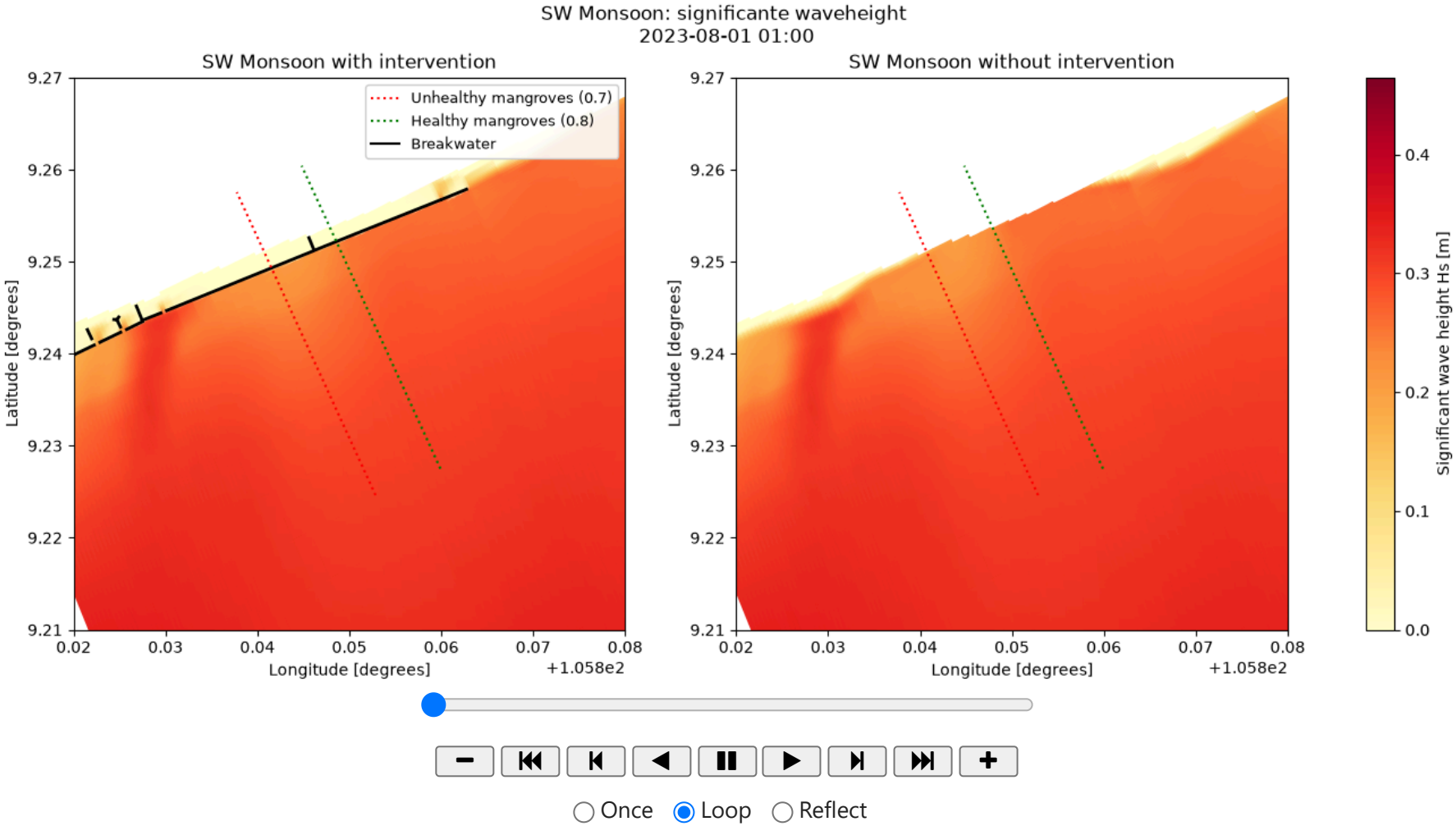


In [12]:
```
flow_difference('sw_intervention', 'sw_baseline', 'SW Monsoon')
```

```
SW Monsoon: 11651 faces, 100 tijdstappen (2023-08-03 00:00 -> 2023-08-27 18:00)
|Δsnelheid| over alle faces en tijden: P50 0.018, P95 0.191, P99 0.419, max 2.132 m/s
Grootste verschil: +2.132 m/s op 2023-08-12 06:00, lon 105.8589, lat 9.2590
```

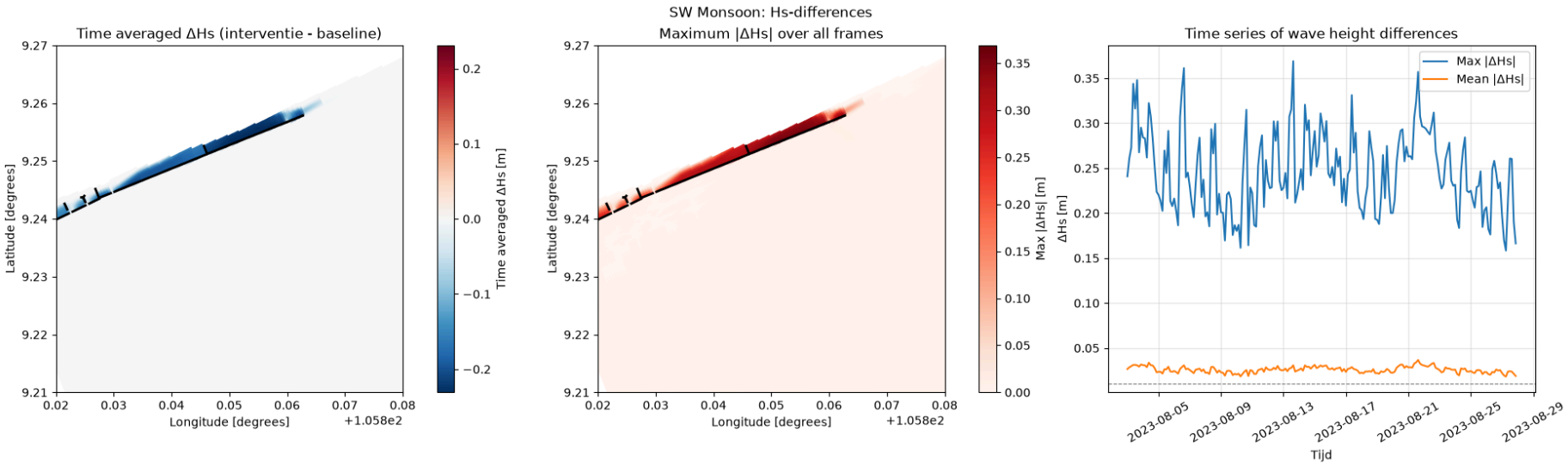


In [13]:
```
sw_hs_anim = hs_comparison_animation('sw_intervention', 'sw_baseline', 'SW Monsoon')
show_anim(sw_hs_anim)
```



In [14]:
```
hs_difference('sw_intervention', 'sw_baseline', 'SW Monsoon')
```

```
--- SW Monsoon ---  (200 tijdstappen, spin-up weggelaten)
Grootste verschil: -0.369 m op 2023-08-13 15:00, lon 105.8612, lat 9.2572
Punten met |ΔHs| > 0.01 m (ooit): 558 van 3189 actieve golfcellen
Gemiddeld |ΔHs| in uitsnede: 0.026 m
```



5. NE- versus SW-monsoon (baseline-flow and intervention -Hs)

In [15]:
```
def residual_compare(name, xlim=None, ylim=None, arrows_nx=30, arrow_len_deg=0.10, pct=97, chunk=48):
    """Residuele (gemiddelde) stroming NE vs SW, baseline. Alle uurlijkse stappen na spin-up (geen sub-sampling)."""
    t0 = time.perf_counter()
    ne, sw = flow('ne_baseline'), flow('sw_baseline')
    assert_same_mesh(ne, sw)
    fx, fy = face_xy(ne)
    xlim = xlim or (float(np.nanmin(fx)), float(np.nanmax(fx)))
    ylim = ylim or (float(np.nanmin(fy)), float(np.nanmax(fy)))
    ny = max(int(arrows_nx * (ylim[1] - ylim[0]) / (xlim[1] - xlim[0])), 4)
    grid = make_arrow_grid(fx, fy, xlim, ylim, nx=arrows_nx, ny=ny)
    sel = grid['sel']
    polys, keep = face_polygons(ne, sel)
    idx = np.arange(SPINUP_H, min(ne.sizes['time'], sw.sizes['time']))
    print(f'{name}: {sel.size} faces, {idx.size} tijdstappen per scenario')

    def mean_uv(ds):
        su, sv, cnt, mx = (np.zeros(sel.size) for _ in range(4))
        for i in range(0, idx.size, chunk):
            ch = idx[i:i + chunk]
            u = ds['mesh2d_ucx'].isel(time=ch, mesh2d_nFaces=sel).values
```

```
            v = ds['mesh2d_ucy'].isel(time=ch, mesh2d_nFaces=sel).values
            ok = np.isfinite(u) & np.isfinite(v)
            su += np.where(ok, u, 0).sum(axis=0)
            sv += np.where(ok, v, 0).sum(axis=0)
            cnt += ok.sum(axis=0)
            mx = np.maximum(mx, np.where(ok, np.hypot(u, v), 0).max(axis=0))
        with np.errstate(invalid='ignore', divide='ignore'):
            return su / cnt, sv / cnt, mx

uN, vN, maxN = mean_uv(ne)
uS, vS, maxS = mean_uv(sw)
print(f'Data gelezen in {time.perf_counter() - t0:.0f} s')
land = (maxN == 0) & (maxS == 0)
uN, vN, uS, vS = (np.where(land, np.nan, a) for a in (uN, vN, uS, vS))
mN, mS = np.hypot(uN, vN), np.hypot(uS, vS)
dU, dV = uS - uN, vS - vN
mD = np.hypot(dU, dV)

for lab, u, v in (('NE', uN, vN), ('SW', uS, vS)):
    mu, mv = np.nanmean(u), np.nanmean(v)
    print(f'{lab}: ruimtelijk gemiddelde residu u={mu:+.4f}, v={mv:+.4f} m/s, grootte={np.hypot(mu, mv):.4f} m/s, '
          f'richting NAAR {np.degrees(np.arctan2(mu, mv)) % 360:.0f}° (t.o.v. noord)')
ok = np.isfinite(uN) & np.isfinite(uS) & (mN > 0.01) & (mS > 0.01)
if ok.sum() > 2:
    cosang = (uN[ok] * uS[ok] + vN[ok] * vS[ok]) / (mN[ok] * mS[ok])
    print(f'Gemiddeld |SW-NE|: {np.nanmean(mD):.4f} m/s | richtingsverschil >90°: {100 * np.mean(cosang < 0):.0f}% '
          f'| correlatie u: {np.corrcoef(uN[ok], uS[ok])[0, 1]:.2f}, v: {np.corrcoef(vN[ok], vS[ok])[0, 1]:.2f}')

with warnings.catch_warnings():
    warnings.simplefilter('ignore')
    both = np.r_[mN[np.isfinite(mN)], mS[np.isfinite(mS)]]
    ref = float(np.nanpercentile(both, 95))
    vmax = float(np.nanpercentile(both, pct))
    dmax = float(np.nanpercentile(mD[np.isfinite(mD)], pct))
fac = arrow_len_deg / ref
print(f'Pijlschaal: {arrow_len_deg}° = {ref:.3f} m/s')

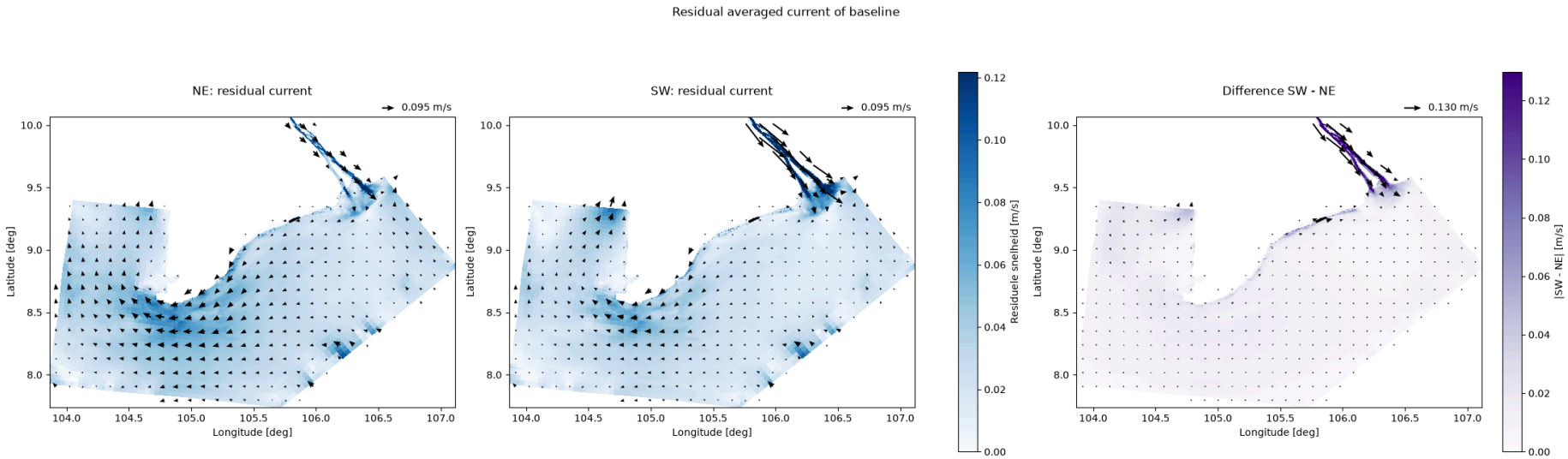
def binned(u, v):
    bu, bv = bin_vectors(grid, u[None], v[None])
    return bu[0], bv[0]
```

```
    def draw(ax, vals, cmap, vmax_, title, u, v, key):
        pc = add_polys(ax, polys, vals[keep], cmap, 0, vmax_)
        bu, bv = binned(u, v)
        q = ax.quiver(grid['x'], grid['y'], bu * fac, bv * fac, angles='xy', scale_units='xy', scale=1,
                      color='black', width=0.0035, headwidth=4, headlength=5)
        ax.quiverkey(q, 0.85, 1.03, key * fac, f'{key:.3f} m/s', labelpos='E', coordinates='axes')
        draw_boundaries(ax)
        ax.set(xlim=xlim, ylim=ylim, aspect='equal', xlabel='Longitude [deg]', ylabel='Latitude [deg]')
        ax.set_title(title, pad=22)
        return pc

    fig, ax = plt.subplots(1, 3, figsize=(22, 7), constrained_layout=True)
    p1 = draw(ax[0], mN, 'Blues', vmax, 'NE: residual current', uN, vN, ref)
    draw(ax[1], mS, 'Blues', vmax, 'SW: residual current', uS, vS, ref)
    fig.colorbar(p1, ax=ax[:2], label='Residuele snelheid [m/s]', shrink=0.8)
    p3 = draw(ax[2], mD, 'Purples', dmax, 'Difference SW - NE', dU, dV, dmax)
    fig.colorbar(p3, ax=ax[2], label='|SW - NE| [m/s]', shrink=0.8)
    fig.suptitle(f'Residual averaged current of baseline')
    plt.show()
    print(f'Totale tijd: {time.perf_counter() - t0:.0f} s')


residual_compare('hele_domein', arrows_nx=30, arrow_len_deg=0.10)
```

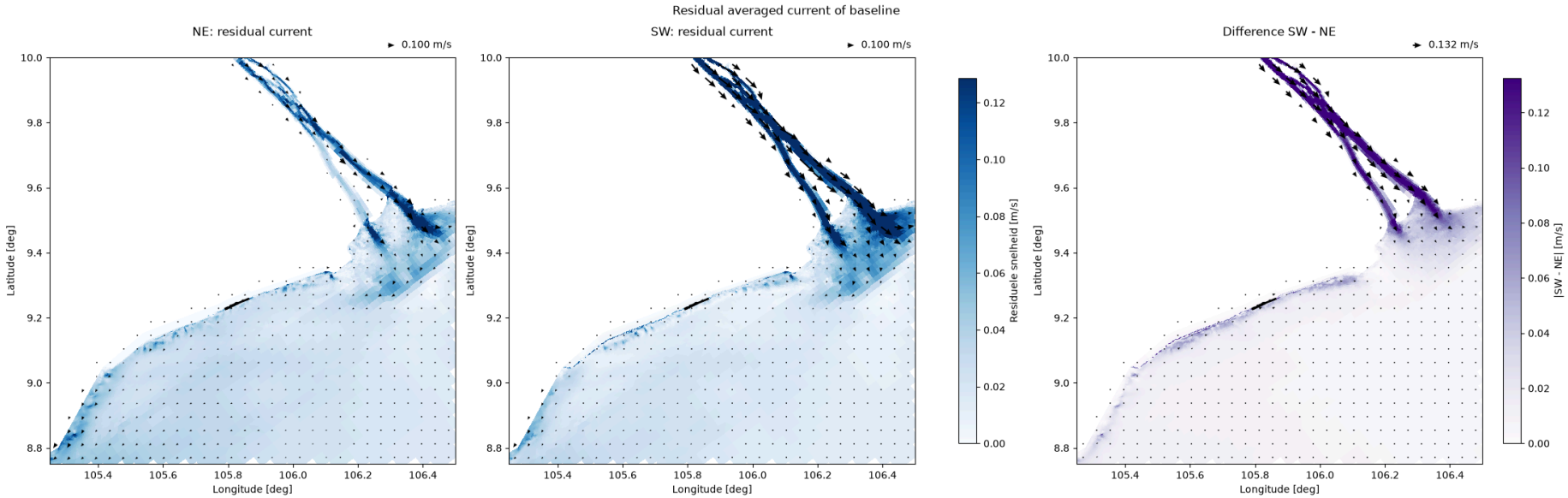
```
hele_domein: 52615 faces, 598 tijdstappen per scenario
Data gelezen in 3 s
NE: ruimtelijk gemiddelde residu u=-0.0189, v=-0.0122 m/s, grootte=0.0225 m/s, richting NAAR 237° (t.o.v. noord)
SW: ruimtelijk gemiddelde residu u=+0.0104, v=-0.0091 m/s, grootte=0.0138 m/s, richting NAAR 131° (t.o.v. noord)
Gemiddeld |SW-NE|: 0.0393 m/s | richtingsverschil >90°: 37% | correlatie u: 0.61, v: 0.63
Pijlschaal: 0.1° = 0.095 m/s
```



Totale tijd: 6 s

In [16]:
```
residual_compare('zoom_nha_mat', xlim=(105.25, 106.5), ylim=(8.75, 10), arrows_nx=30, arrow_len_deg=0.02)
```

```
zoom_nha_mat: 46097 faces, 598 tijdstappen per scenario
Data gelezen in 531 s
NE: ruimtelijk gemiddelde residu u=-0.0181, v=-0.0138 m/s, grootte=0.0228 m/s, richting NAAR 233° (t.o.v. noord)
SW: ruimtelijk gemiddelde residu u=+0.0138, v=-0.0102 m/s, grootte=0.0172 m/s, richting NAAR 126° (t.o.v. noord)
Gemiddeld |SW-NE|: 0.0428 m/s | richtingsverschil >90°: 43% | correlatie u: 0.60, v: 0.61
Pijlschaal: 0.02° = 0.100 m/s
```



Totale tijd: 533 s

In [17]:
```
def structured_triangulation(x, y, active):
    """Triangulatie op de (nmax, mmax)-connectiviteit van het golfrooster; inactieve cellen gemaskeerd."""
    ny, nx = x.shape
    idx = np.arange(ny * nx).reshape(ny, nx)
    a, b = idx[:-1, :-1].ravel(), idx[:-1, 1:].ravel()
    c, d = idx[1:, 1:].ravel(), idx[1:, :-1].ravel()
    tris = np.concatenate([np.column_stack((a, b, c)), np.column_stack((a, c, d))])
    ok = (active & np.isfinite(x) & np.isfinite(y)).ravel()
    bad = ~(ok[a] & ok[b] & ok[c] & ok[d])
    # Niet-eindige coördinaten NIET op 0 zetten (rekt de as-limieten uit naar (0,0)):
    # vul ze met het domeingemiddelde, dat binnen de bounding box blijft.
    fin = np.isfinite(x) & np.isfinite(y)
    xf = np.where(fin, x, np.mean(x[fin])).ravel()
    yf = np.where(fin, y, np.mean(y[fin])).ravel()
    return Triangulation(xf, yf, tris, mask=np.concatenate([bad, bad]))


def hs_compare_domain(name, stride=3, pct=90):
    """Periode-gemiddelde Hs voor NE en SW (interventie-runs) en het verschil SW - NE."""
    t0 = time.perf_counter()
    ne, sw = wave('ne_intervention'), wave('sw_intervention')
```

```
    x, y = ne['x'].values, ne['y'].values
    if not (np.array_equal(x, sw['x'].values, equal_nan=True) and np.array_equal(y, sw['y'].values, equal_nan=True)):
        raise ValueError('NE en SW golfroosters verschillen')
    idx = np.arange(SPINUP_H, min(ne.sizes['time'], sw.sizes['time']), stride)
    print(f'{name}: {idx.size} tijdstappen')

    hs_ne = ne['hsign'].isel(time=idx).mean('time', skipna=True).values
    hs_sw = sw['hsign'].isel(time=idx).mean('time', skipna=True).values   # .mean was weg in je versie
    active = (ne['kcs'].values > 0) & (sw['kcs'].values > 0) & np.isfinite(hs_ne) & np.isfinite(hs_sw) \
             & np.isfinite(x) & np.isfinite(y)
    dif = hs_sw - hs_ne

    with warnings.catch_warnings():
        warnings.simplefilter('ignore')
        vmax = float(np.nanpercentile(np.r_[hs_ne[active], hs_sw[active]], pct))
        dmax = float(np.nanpercentile(np.abs(dif[active]), pct))
    print(f'Hs-schaal 0 - {vmax:.3f} m | verschilschaal ±{dmax:.3f} m')

    tri = structured_triangulation(x, y, active)
    xl = (float(x[active].min()), float(x[active].max()))     # expliciete limieten uit actieve cellen
    yl = (float(y[active].min()), float(y[active].max()))

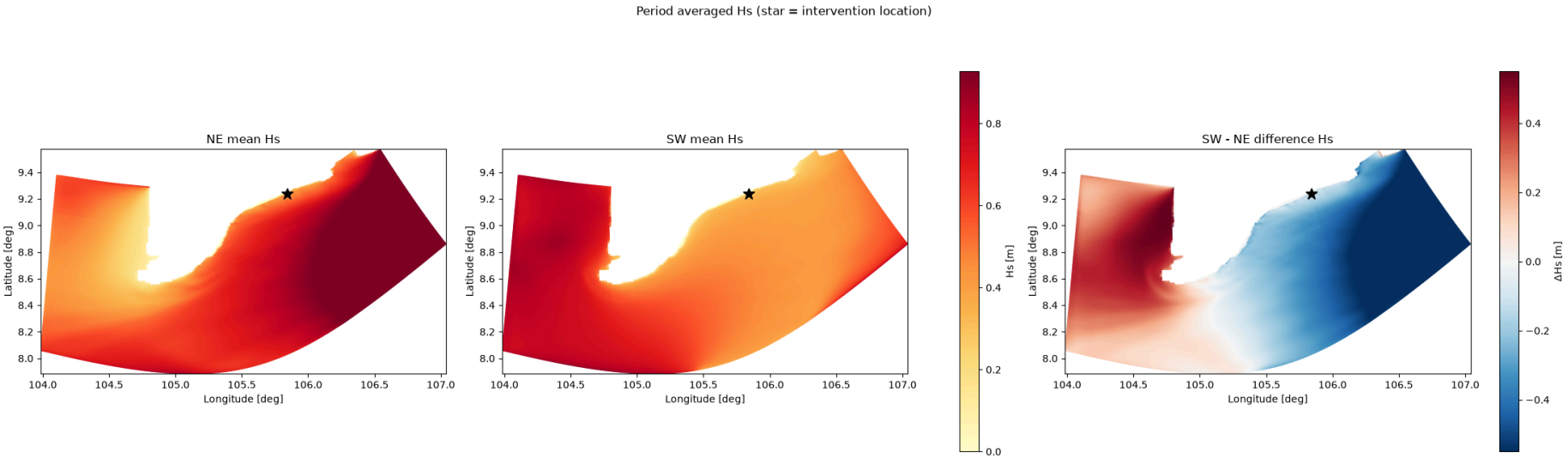
    fig, ax = plt.subplots(1, 3, figsize=(22, 7), constrained_layout=True)
    kw = dict(shading='gouraud')
    im1 = ax[0].tripcolor(tri, np.nan_to_num(hs_ne).ravel(), cmap='YlOrRd', vmin=0, vmax=vmax, **kw)
    ax[1].tripcolor(tri, np.nan_to_num(hs_sw).ravel(), cmap='YlOrRd', vmin=0, vmax=vmax, **kw)
    im3 = ax[2].tripcolor(tri, np.nan_to_num(dif).ravel(), cmap='RdBu_r', vmin=-dmax, vmax=dmax, **kw)

    for a, t in zip(ax, ('NE mean Hs', 'SW mean Hs', 'SW - NE difference Hs')):
        a.plot(X_CENTER, Y_CENTER, marker='*', color='black', markersize=12, linestyle='')
        a.set(xlim=xl, ylim=yl, aspect='equal', xlabel='Longitude [deg]', ylabel='Latitude [deg]', title=t)

    fig.colorbar(im1, ax=ax[:2], label='Hs [m]', shrink=0.8)
    fig.colorbar(im3, ax=ax[2], label='ΔHs [m]', shrink=0.8)
    fig.suptitle('Period averaged Hs (star = intervention location)')
    plt.show()
    print(f'Totale tijd: {time.perf_counter() - t0:.0f} s')


hs_compare_domain('hele_domein')
```

```
hele_domein: 208 tijdstappen
Hs-schaal 0 - 0.927 m | verschilschaal ±0.550 m
```




```
Totale tijd: 1 s
```

6. Storm scenario (15-19 augustus 2023)

In [18]:
```python
def storm_load(stride=2, arrows_nx=26):
    fl, wv = flow('storm'), open_ds(WAVE_FILES['storm'])
    print('Flow dims:', dict(fl.sizes), '| mesh2d_s1 aanwezig:', 'mesh2d_s1' in fl.variables)
    common, iif, iiw = common_times(fl, wv, stride=stride)
    fx, fy = face_xy(fl)
    polys, keep = face_polygons(fl, np.arange(fx.size))
    fxk, fyk = fx[keep], fy[keep]
    u = fl['mesh2d_ucx'].isel(time=iif).values[:, keep]
    v = fl['mesh2d_ucy'].isel(time=iif).values[:, keep]
    spd = np.hypot(u, v)
    has_s1 = 'mesh2d_s1' in fl.variables
    s1 = fl['mesh2d_s1'].isel(time=iif).values[:, keep] if has_s1 else None

    nxn, nyn = fl['mesh2d_node_x'].values, fl['mesh2d_node_y'].values
    xlim, ylim = (float(np.nanmin(nxn)), float(np.nanmax(nxn))), (float(np.nanmin(nyn)), float(np.nanmax(nyn)))
    ny = max(int(arrows_nx * (ylim[1] - ylim[0]) / (xlim[1] - xlim[0])), 4)
    grid = make_arrow_grid(fxk, fyk, xlim, ylim, nx=arrows_nx, ny=ny)
```

```
ub, vb = bin_vectors(grid, u[:, grid['sel']], v[:, grid['sel']])

x, y = wv['x'].values, wv['y'].values
active = np.isfinite(x) & np.isfinite(y) & (wv['kcs'].values > 0)
hs = wv['hsign'].isel(time=iiw).values
tri = structured_triangulation(x, y, active)

# Dichtstbijzijnde flow-face en golfcel bij de interventielocatie
cosl = np.cos(np.deg2rad(Y_CENTER))
f_near = int(np.argmin(np.hypot((fxk - X_CENTER) * cosl, fyk - Y_CENTER)))
wd = np.where(active, np.hypot((x - X_CENTER) * cosl, y - Y_CENTER), np.inf)
w_near = np.unravel_index(int(np.argmin(wd)), wd.shape)
hs_site = hs[:, w_near[0], w_near[1]]
hs_dom = np.nanmax(np.where(active[None], hs, np.nan), axis=(1, 2))

D = dict(common=common, polys=polys, keep=keep, spd=spd, ub=ub, vb=vb, grid=grid, s1=s1, tri=tri, hs=hs,
         active=active, f_near=f_near, hs_site=hs_site, hs_dom=hs_dom, xlim=xlim, ylim=ylim, surge=None)

# Waterstand-opzet t.o.v. de SW-baseline (zelfde periode, zelfde getij, geen storm) - indicatief
if has_s1:
    try:
        ref = flow('sw_baseline')
        both, i_s, i_r = np.intersect1d(common, ref['time'].values, return_indices=True)
        tree, ok = face_tree(ref)
        _, k = tree.query([[fxk[f_near], fyk[f_near]]])
        s1_ref = ref['mesh2d_s1'].isel(time=i_r, mesh2d_nFaces=int(ok[k[0]])).values
        D['surge'] = (both, s1[i_s, f_near] - s1_ref)
    except FileNotFoundError:
        pass

print(f'{common.size} frames: {time_str(common[0])} -> {time_str(common[-1])}')
print(f'Dichtstbijzijnde flow-face: {np.hypot(fxk[f_near] - X_CENTER, fyk[f_near] - Y_CENTER):.3f}° van de interventie; '
      f'golfcel: {np.sqrt(wd[w_near]**2):.3f}° (golfrooster grof)')
print(f'Max Hs domein: {np.nanmax(hs_dom):.2f} m op {time_str(common[int(np.nanargmax(hs_dom))])}')
print(f'Max snelheid domein: {np.nanmax(spd):.2f} m/s op {time_str(common[int(np.nanargmax(np.nanmax(spd, axis=1)))])}')
print(f'Bij interventie: max Hs {np.nanmax(hs_site):.2f} m (op {time_str(common[int(np.nanargmax(hs_site))])}), '
      f'max snelheid {np.nanmax(spd[:, f_near]):.2f} m/s')
if has_s1:
    print(f'Waterstand bij interventie: max {np.nanmax(s1[:, f_near]):.2f} m op {time_str(common[int(np.nanargmax(s1[:, f_
if D['surge'] is not None:
    print(f'Indicatieve opzet t.o.v. SW-baseline: max {np.nanmax(D["surge"][1]):+.2f} m '
```

```
            f'(verschil tussen twee verschillende runs/meshes; geen zuivere surge)')
    return D


def storm_base(ax, D, title):
    for _, bx, by in BOUNDARIES_ALL:
        ax.plot(bx, by, color='black', linewidth=0.4, alpha=0.7)
    ax.plot(X_CENTER, Y_CENTER, marker='*', color='red', markersize=14, markeredgecolor='black', linestyle='')
    ax.set(xlim=D['xlim'], ylim=D['ylim'], aspect='equal', xlabel='Longitude [deg]', ylabel='Latitude [deg]')
    ax.set_title(title)


def storm_summary_plot(D):
    tt, fn = D['common'], D['f_near']
    vmax_s = float(np.nanpercentile(D['spd'], 99.5))
    vmax_h = float(np.nanmax(D['hs_dom']))
    fig, ax = plt.subplots(2, 3, figsize=(21, 11), constrained_layout=True)
    p = add_polys(ax[0, 0], D['polys'], np.nanmax(D['spd'], axis=0), 'Blues', 0, vmax_s)
    storm_base(ax[0, 0], D, 'Max flow velocity during storm')
    fig.colorbar(p, ax=ax[0, 0], shrink=0.8, label='[m/s]')
    hmax_map = np.nanmax(np.where(D['active'][None], D['hs'], np.nan), axis=0)
    q = ax[0, 1].tripcolor(D['tri'], np.nan_to_num(hmax_map).ravel(), shading='gouraud', cmap='YlOrRd', vmin=0, vmax=vmax_h)
    storm_base(ax[0, 1], D, 'Maximale Hs during storm')
    fig.colorbar(q, ax=ax[0, 1], shrink=0.8, label='[m]')
    ax[0, 2].plot(tt, D['hs_dom'], label='Max Hs domain')
    ax[0, 2].plot(tt, D['hs_site'], label='Hs at intervention')
    ax[0, 2].set(ylabel='Hs [m]', title='Significant waveheight Hs')
    ax[0, 2].legend()
    ax[1, 0].plot(tt, np.nanmax(D['spd'], axis=1), label='Max velocity domain')
    ax[1, 0].plot(tt, D['spd'][:, fn], label='Velocity at intervention')
    ax[1, 0].set(ylabel='Velocity [m/s]', title='Flow velocity')
    ax[1, 0].legend()
    if D['s1'] is not None:
        ax[1, 1].plot(tt, D['s1'][:, fn], color='tab:green')
        ax[1, 1].set(ylabel='Waterlevel [m]', title='Waterlevel at intervention (mesh2d_s1)')
    if D['surge'] is not None:
        ax[1, 2].plot(*D['surge'], color='tab:purple')
        ax[1, 2].axhline(0, color='grey', linewidth=0.8)
        ax[1, 2].set(ylabel='Difference [m]', title='Waterlevel storm-run minus SW-baseline (indicative)')
    else:
        ax[1, 2].axis('off')
```

```
    for a in (ax[0, 2], ax[1, 0], ax[1, 1], ax[1, 2]):
        a.grid(alpha=0.3)
        a.tick_params(axis='x', rotation=30)
    fig.suptitle('Storm: max')
    plt.show()

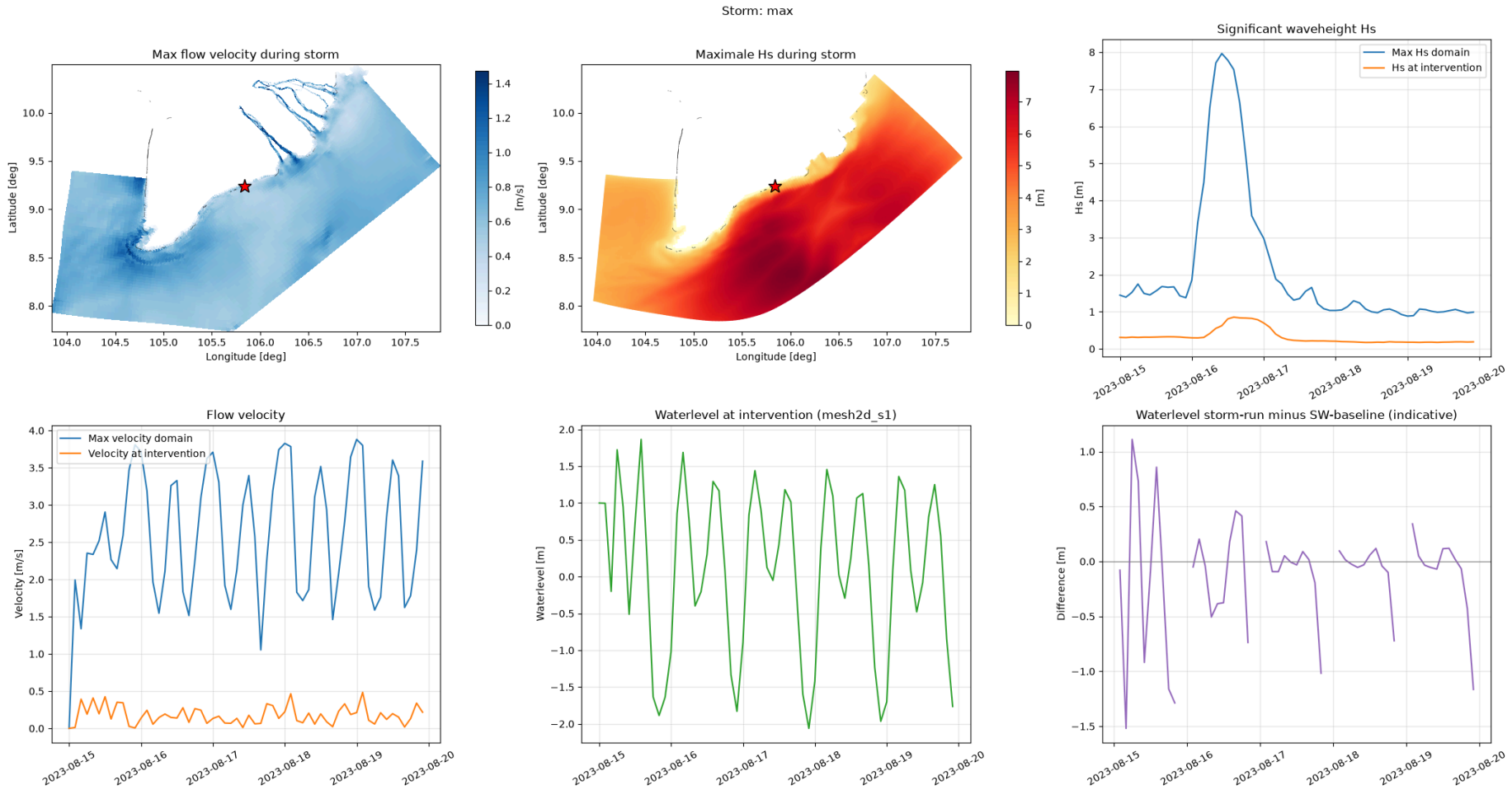

def storm_animation(D, arrow_len=0.12):
    tt, spd = D['common'], D['spd']
    ref = float(np.nanpercentile(spd, 95))
    fac = arrow_len / ref
    vmax_s = float(np.nanpercentile(spd, 99.5))
    vmax_h = float(np.nanmax(D['hs_dom']))
    g = D['grid']
    fig, ax = plt.subplots(1, 2, figsize=(16, 7), dpi=55, constrained_layout=True)
    pc = add_polys(ax[0], D['polys'], spd[0], 'Blues', 0, vmax_s)
    qv = ax[0].quiver(g['x'], g['y'], D['ub'][0] * fac, D['vb'][0] * fac, angles='xy', scale_units='xy', scale=1,
                      color='black', width=0.003)
    storm_base(ax[0], D, 'Flow speed en richting')
    fig.colorbar(pc, ax=ax[0], shrink=0.8, label='[m/s]')
    hc = ax[1].tripcolor(D['tri'], np.nan_to_num(D['hs'][0]).ravel(), shading='gouraud', cmap='YlOrRd', vmin=0, vmax=vmax_h)
    storm_base(ax[1], D, 'Significante golfhoogte Hs')
    fig.colorbar(hc, ax=ax[1], shrink=0.8, label='[m]')
    ttl = fig.suptitle(f'Storm {time_str(tt[0])}')

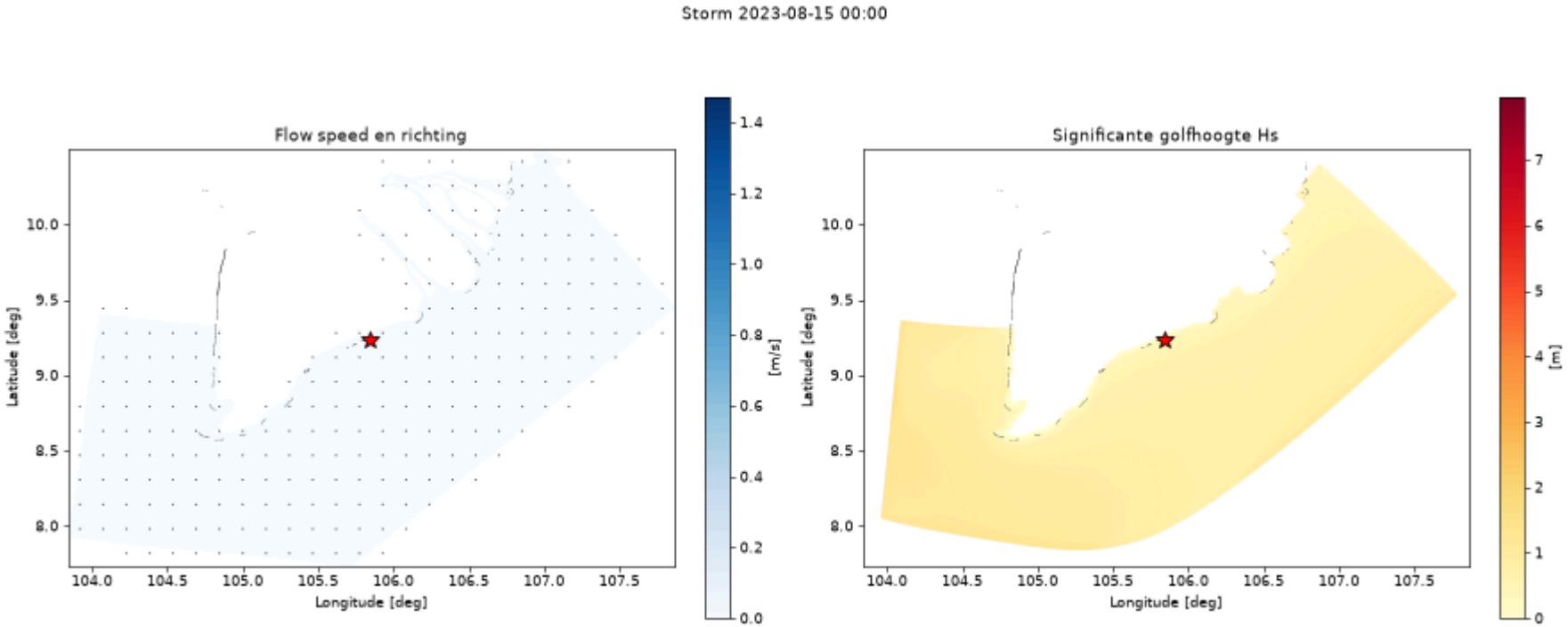
    def update(k):
        pc.set_array(np.ma.masked_invalid(spd[k]))
        qv.set_UVC(D['ub'][k] * fac, D['vb'][k] * fac)
        hc.set_array(np.nan_to_num(D['hs'][k]).ravel())
        ttl.set_text(f'Storm {time_str(tt[k])}')
        return pc, qv, hc, ttl

    anim = FuncAnimation(fig, update, frames=len(tt), interval=200, blit=False)
    plt.close(fig)
    return anim


storm = storm_load(stride=2)
storm_summary_plot(storm)
show_anim(storm_animation(storm))
```

```
storm: Storm\FlowFM_map.nc
Flow dims: {'mesh2d_nNodes': 50266, 'mesh2d_nEdges': 104514, 'Two': 2, 'mesh2d_nFaces': 54229, 'mesh2d_nMax_face_nodes': 4, 'time': 120} | mesh2d_s1 aanwezig: True
60 frames: 2023-08-15 00:00 -> 2023-08-19 22:00
Dichtstbijzijnde flow-face: 0.006° van de interventie; golfcel: 0.012° (golfrooster grof)
Max Hs domein: 7.96 m op 2023-08-16 10:00
Max snelheid domein: 3.88 m/s op 2023-08-19 00:00
Bij interventie: max Hs 0.86 m (op 2023-08-16 14:00), max snelheid 0.48 m/s
Waterstand bij interventie: max 1.86 m op 2023-08-15 14:00
Indicatieve opzet t.o.v. SW-baseline: max +1.11 m (verschil tussen twee verschillende runs/meshes; geen zuivere surge)
C:\Users\Larsk\AppData\Local\Temp\ipykernel_4796\4141034815.py:79: RuntimeWarning: All-NaN slice encountered
  hmax_map = np.nanmax(np.where(D['active'][None], D['hs'], np.nan), axis=0)
```





7. Transects healthy and unhealthy mangroves and diagnostics

7.1 Flow-profiles (NE en SW)

In [19]:
```
def flow_transect_profile(ds, fraction, time_idx=None):
    """Snelheid en diepte langs het transect (dichtstbijzijnde face per punt).
    time_idx: None (alles na spin-up), array van tijdindexen of één integer."""
    s, xy = transect_xy(fraction)
    lon, lat = to_lonlat(xy)
    tree, ok = face_tree(ds)                              # KD-tree: één keer per dataset
```

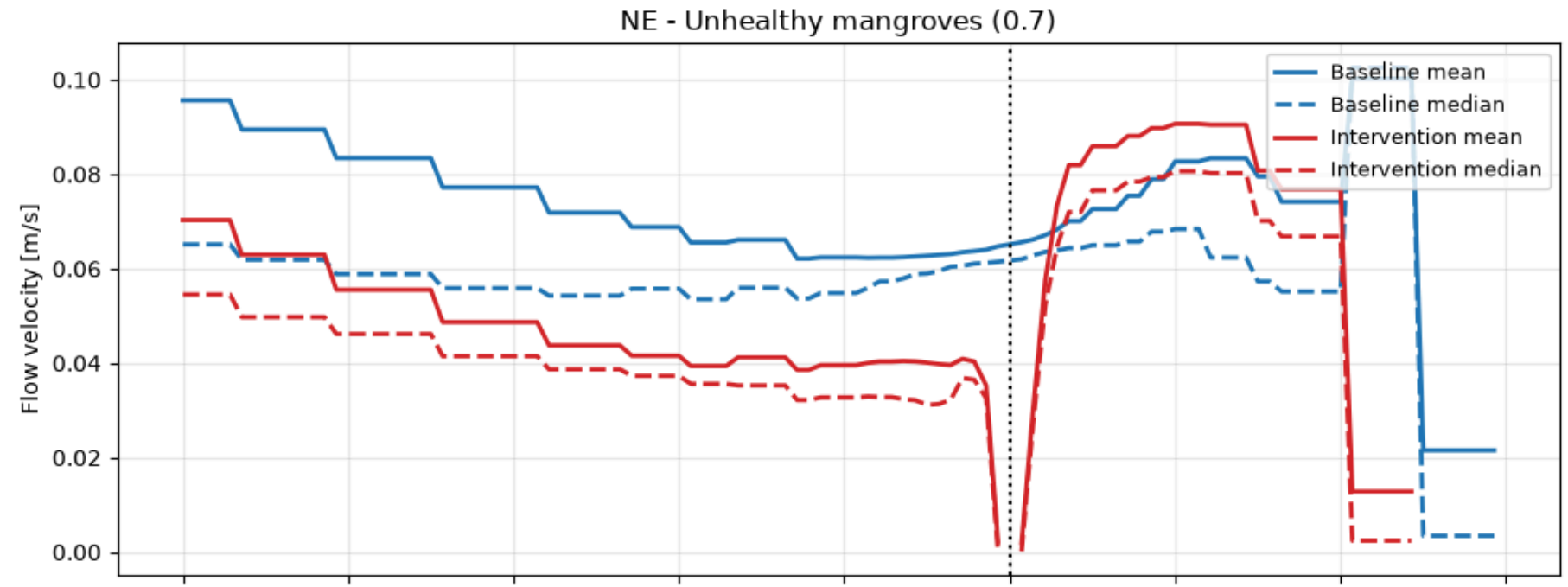
```
    d, k = tree.query(np.column_stack((lon, lat)))
    faces = ok[k]
    uniq, inv = np.unique(faces, return_inverse=True)
    t_sel = slice(SPINUP_H, None) if time_idx is None else time_idx
    pick = dict(time=t_sel, mesh2d_nFaces=uniq)
    u = np.atleast_2d(ds['mesh2d_ucx'].isel(**pick).values)
    v = np.atleast_2d(ds['mesh2d_ucy'].isel(**pick).values)
    depth = np.atleast_2d(ds['mesh2d_waterdepth'].isel(**pick).values)
    speed = np.hypot(u, v)[:, inv]
    depth = depth[:, inv]
    speed[depth < 0.01] = np.nan                          # droog / vrijwel droog
    with warnings.catch_warnings():
        warnings.simplefilter('ignore', RuntimeWarning)
        always_dry = np.nanmax(depth, axis=0) <= 0
    speed[:, always_dry | (d > MAX_SNAP_DEG)] = np.nan
    return dict(s=s, speed=speed, depth=depth, faces=faces, snap=d)

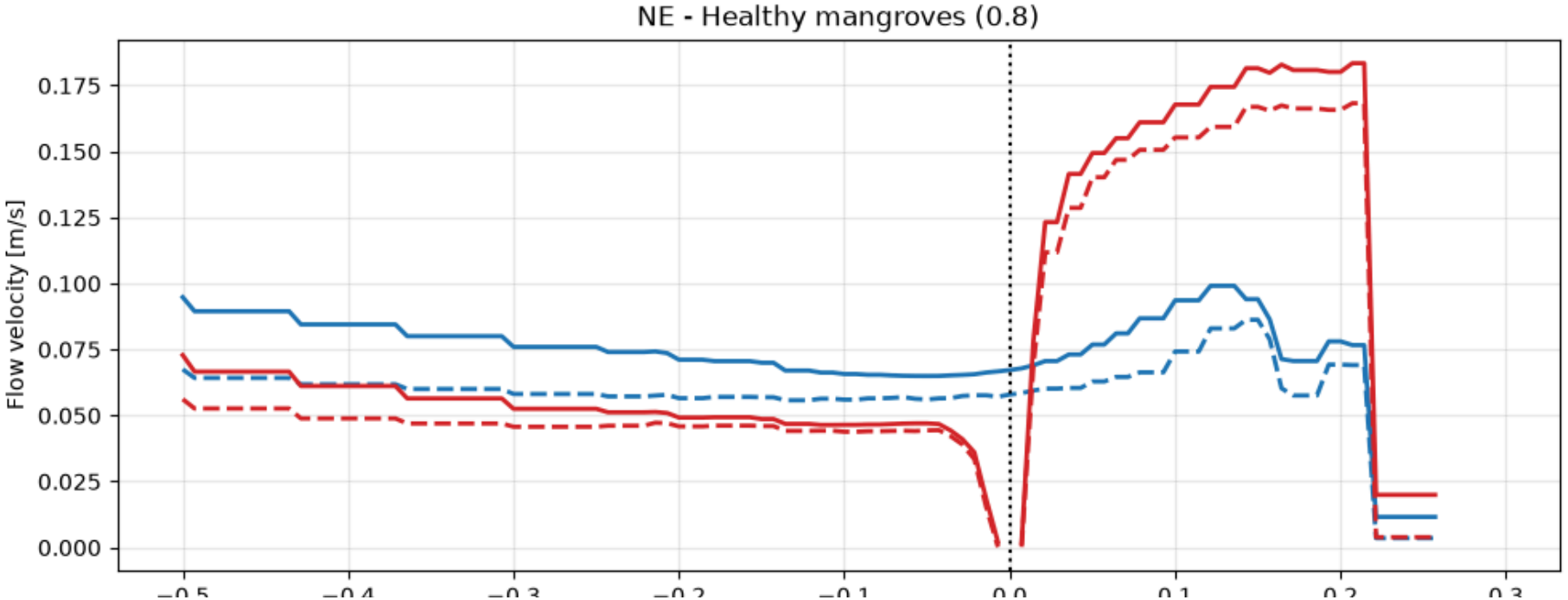

def plot_flow_profiles(key_i, key_b, label):
    ds_i, ds_b = flow(key_i), flow(key_b)
    _, ii, ib = common_times(ds_i, ds_b, start=SPINUP_H)    # alleen gemeenschappelijke tijden
    fig, axes = plt.subplots(len(TRANSECTS), 1, figsize=(10, 8), sharex=True)
    for ax, (frac, (name, _)) in zip(np.atleast_1d(axes), TRANSECTS.items()):
        pb = flow_transect_profile(ds_b, frac, ib)
        pi = flow_transect_profile(ds_i, frac, ii)
        print(f'{label} {name}: {np.unique(pi["faces"]).size} unieke faces onder {TRANSECT_N} punten '
              f'(plateaus = zelfde face), max snap-afstand {pi["snap"].max():.4f}°')
        with warnings.catch_warnings():
            warnings.simplefilter('ignore', RuntimeWarning)
            curves = [('Baseline mean', pb['s'], np.nanmean(pb['speed'], 0), '-', 'tab:blue'),
                      ('Baseline median', pb['s'], np.nanmedian(pb['speed'], 0), '--', 'tab:blue'),
                      ('Intervention mean', pi['s'], np.nanmean(pi['speed'], 0), '-', 'tab:red'),
                      ('Intervention median', pi['s'], np.nanmedian(pi['speed'], 0), '--', 'tab:red')]
        for lab, s, yv, ls, col in curves:
            ax.plot(s, yv, linestyle=ls, color=col, linewidth=2, label=lab)
        ax.axvline(0, color='black', linestyle=':', linewidth=1.5)
        ax.set_title(f'{label} - {name}')
        ax.set_ylabel('Flow velocity [m/s]')
        ax.grid(alpha=0.3)
    np.atleast_1d(axes)[0].legend(loc='upper right', fontsize=9)
    np.atleast_1d(axes)[-1].set_xlabel('Distance from breakwater [km] (negatief = offshore, positief = lee)')
```

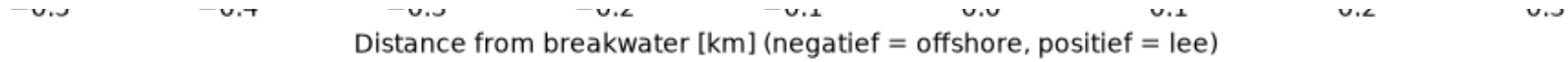
```
    plt.tight_layout()
    plt.show()


plot_flow_profiles('ne_intervention', 'ne_baseline', 'NE')
plot_flow_profiles('sw_intervention', 'sw_baseline', 'SW')
```

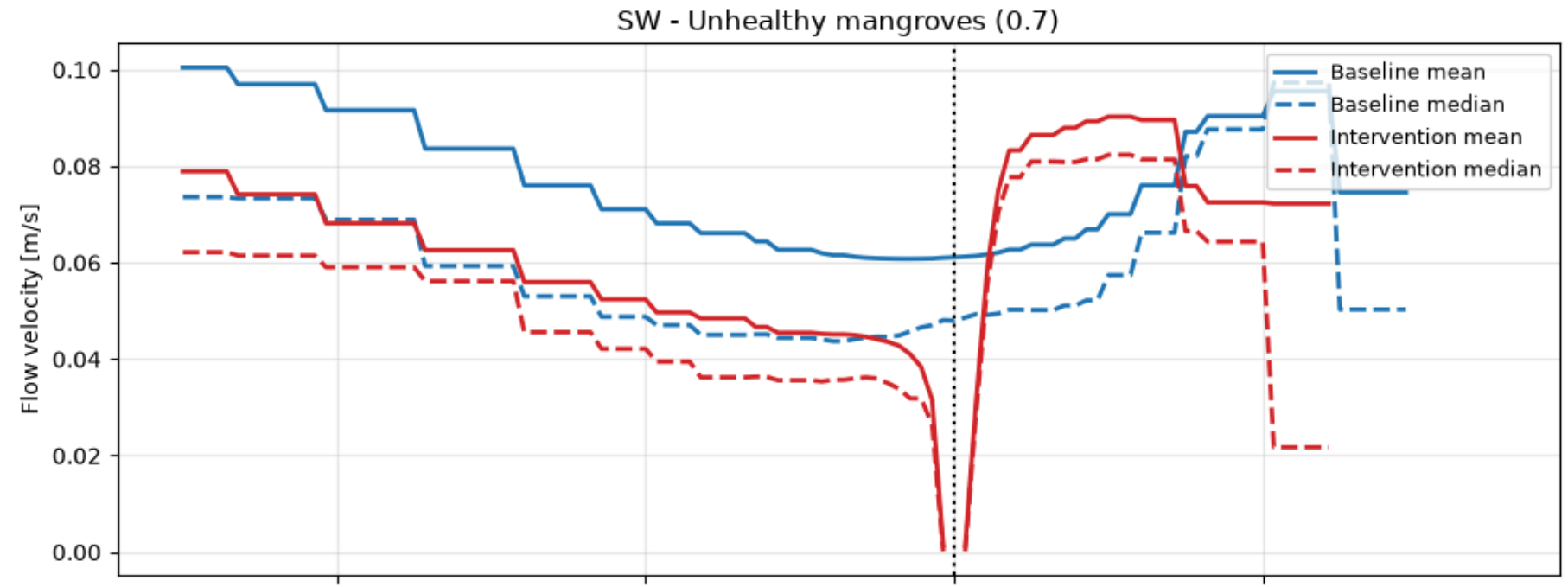
```
NE Unhealthy mangroves (0.7): 38 unieke faces onder 120 punten (plateaus = zelfde face), max snap-afstand 0.0003°
NE Healthy mangroves (0.8): 44 unieke faces onder 120 punten (plateaus = zelfde face), max snap-afstand 0.0003°
```

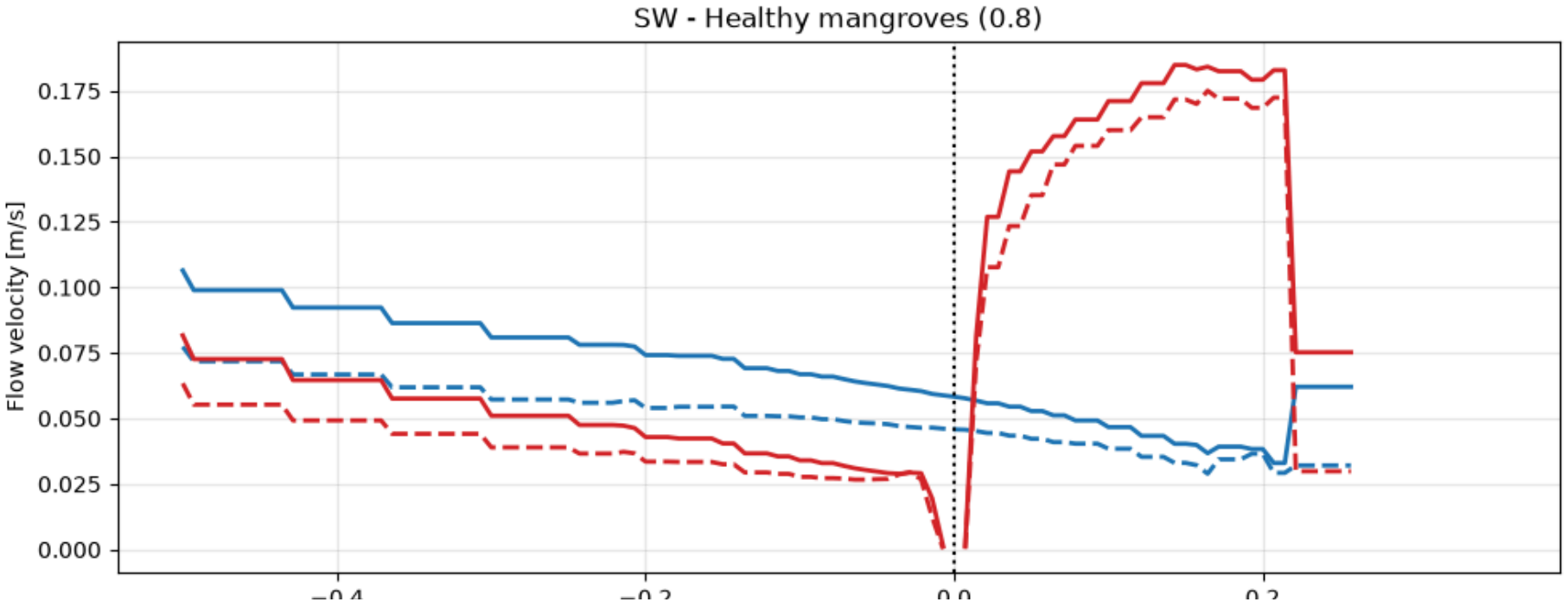

NE - Unhealthy mangroves (0.7)
Flow velocity [m/s]
0.10
0.08
0.06
0.04
0.02
0.00
Baseline mean
Baseline median
Intervention mean
Intervention median


NE - Healthy mangroves (0.8)
Flow velocity [m/s]
0.175
0.150
0.125
0.100
0.075
0.050
0.025
0.000
−0.5
−0.4
−0.3
−0.2
−0.1
0.0
0.1
0.2
0.3




```
SW Unhealthy mangroves (0.7): 38 unieke faces onder 120 punten (plateaus = zelfde face), max snap-afstand 0.0003°
SW Healthy mangroves (0.8): 44 unieke faces onder 120 punten (plateaus = zelfde face), max snap-afstand 0.0003°
```


SW - Unhealthy mangroves (0.7)
Flow velocity [m/s]
0.10
0.08
0.06
0.04
0.02
0.00
Baseline mean
Baseline median
Intervention mean
Intervention median


SW - Healthy mangroves (0.8)
Flow velocity [m/s]
0.175
0.150
0.125
0.100
0.075
0.050
0.025
0.000



7.2 Hs-profiles (NE en SW)

X-as is the geometric distance from the breakwater. The location is where the steepest descend takes place from the intervention profile, because of the grid size this is is not location has a deviaton of a few dozen meters.

In [20]:
```
def wave_tree(ds):
    key = id(ds)
    if key not in _WAVE_TREE:
        x, y = ds['x'].values, ds['y'].values
        active = (ds['kcs'].values > 0) & np.isfinite(x) & np.isfinite(y)
        flat = np.flatnonzero(active.ravel())
        _WAVE_TREE[key] = (cKDTree(np.column_stack((x.ravel()[flat], y.ravel()[flat]))), flat, x.shape)
    return _WAVE_TREE[key]


def hs_transect_profile(ds, fraction, time_idx=None):
    s, xy = transect_xy(fraction)
    lon, lat = to_lonlat(xy)
    tree, flat, shape = wave_tree(ds)
    d, k = tree.query(np.column_stack((lon, lat)))
    rows, cols = np.unravel_index(flat[k], shape)
    t_sel = slice(SPINUP_H, None) if time_idx is None else time_idx
    hs = ds['hsign'].isel(time=t_sel, nmax=xr.DataArray(rows, dims='p'), mmax=xr.DataArray(cols, dims='p')).values
    hs = np.atleast_2d(hs).astype(float)
    hs[:, d > MAX_SNAP_WAVE_DEG] = np.nan
    return dict(s=s, hs=hs, snap=d)


def plot_hs_profiles(key_i, key_b, label):
    ds_i, ds_b = wave(key_i), wave(key_b)
    _, ii, ib = common_times(ds_i, ds_b, start=SPINUP_H)
    fig, axes = plt.subplots(len(TRANSECTS), 1, figsize=(10, 8), sharex=True)

    for ax, (frac, (name, _)) in zip(np.atleast_1d(axes), TRANSECTS.items()):
        pb = hs_transect_profile(ds_b, frac, ib)
        pi = hs_transect_profile(ds_i, frac, ii)
```

```
    with warnings.catch_warnings():
        warnings.simplefilter('ignore', RuntimeWarning)
        mean_i, med_i = np.nanmean(pi['hs'], 0), np.nanmedian(pi['hs'], 0)
        mean_b, med_b = np.nanmean(pb['hs'], 0), np.nanmedian(pb['hs'], 0)

    # Bepaal de steilste daling (breakwater) van de interventie
    grad = np.gradient(mean_i, pi['s'])
    if np.isfinite(grad).any():
        s_drop_idx = int(np.nanargmin(grad))
        s_drop = pi['s'][s_drop_idx]
        # Verschuif de s-as zodat de steilste daling precies op 0 komt te liggen
        s_shifted = pi['s'] - s_drop
        print(f"{label} {name}: steilste Hs-daling bij s = {s_drop * 1000:+.0f} m")
    else:
        s_shifted = pi['s']

    curves = [
        ('Baseline mean', mean_b, '-', 'tab:blue'),
        ('Baseline median', med_b, '--', 'tab:blue'),
        ('Intervention mean', mean_i, '-', 'tab:red'),
        ('Intervention median', med_i, '--', 'tab:red')
    ]

    for lab, yv, ls, col in curves:
        ax.plot(s_shifted, yv, linestyle=ls, color=col, linewidth=2, label=lab)

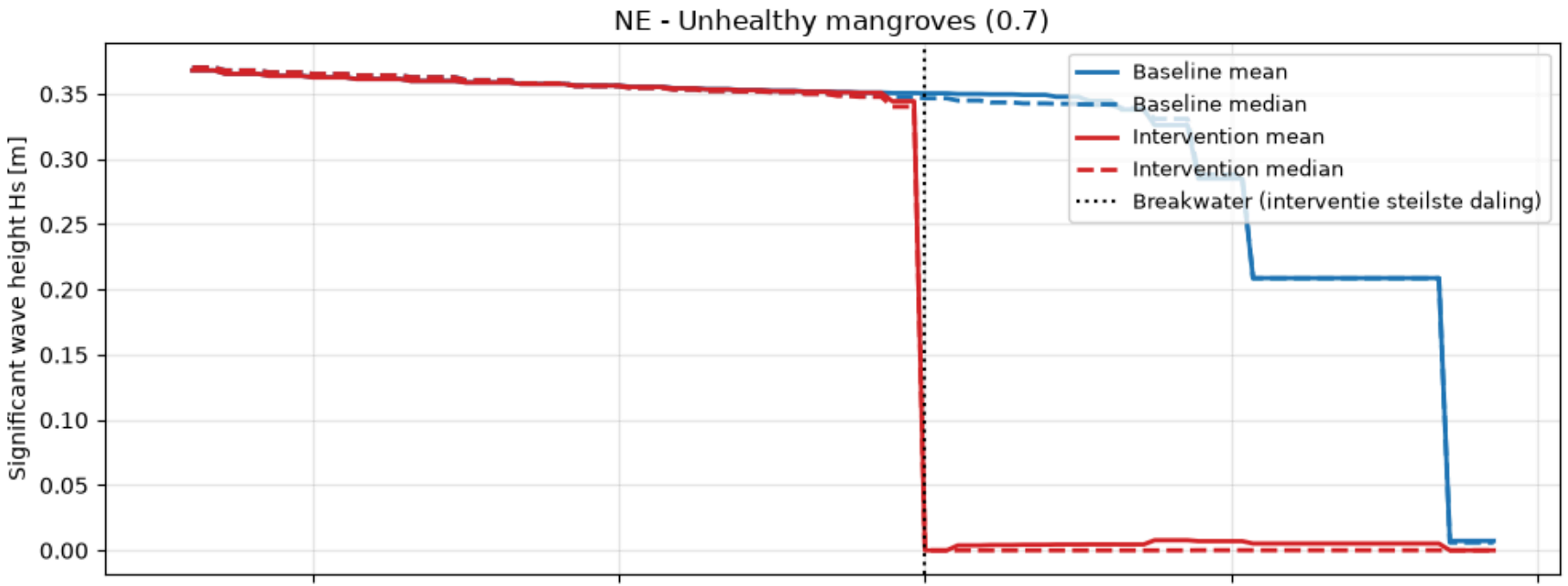
    # Breakwater lijn op s = 0 (gebaseerd op de steilste daling van de interventie)
    ax.axvline(0, color='black', linestyle=':', linewidth=1.5, label='Breakwater (interventie steilste daling)')

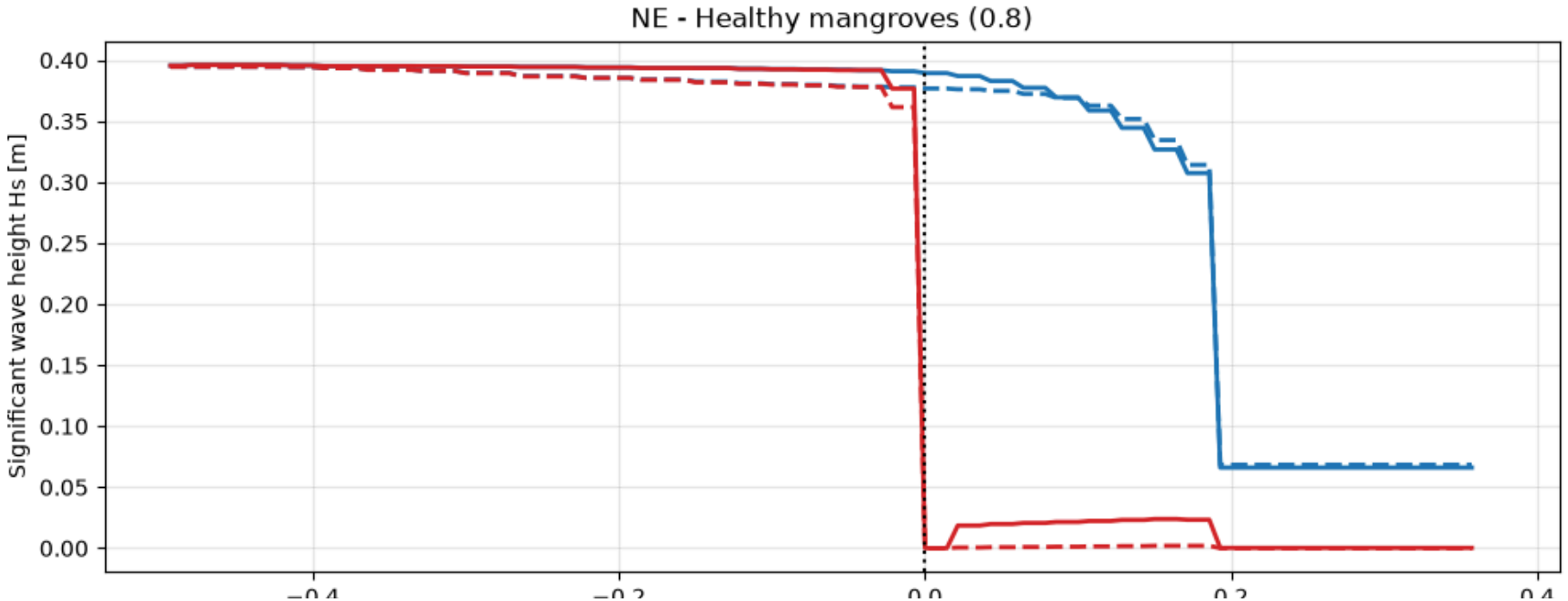
    ax.set_title(f"{label} - {name}")
    ax.set_ylabel('Significant wave height Hs [m]')
    ax.grid(alpha=0.3)

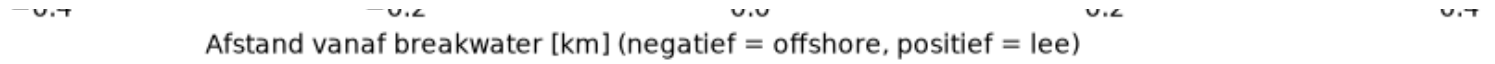
np.atleast_1d(axes)[0].legend(loc='upper right', fontsize=9)
np.atleast_1d(axes)[-1].set_xlabel('Afstand vanaf breakwater [km] (negatief = offshore, positief = lee)')
plt.tight_layout()
plt.show()
```

```
plot_hs_profiles('ne_intervention', 'ne_baseline', 'NE')
plot_hs_profiles('sw_intervention', 'sw_baseline', 'SW')
```

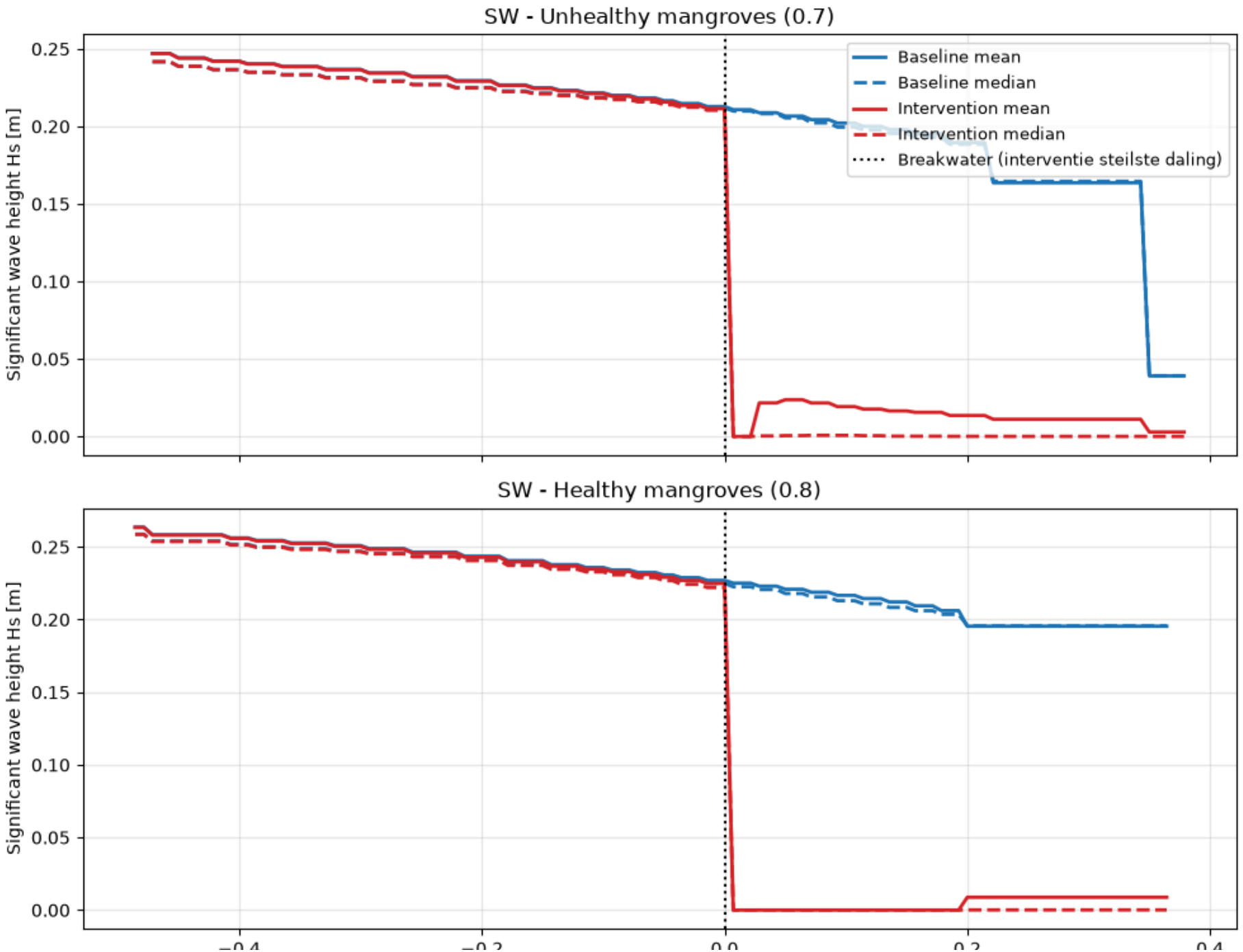
```
NE Unhealthy mangroves (0.7): steilste Hs-daling bij s = -21 m
NE Healthy mangroves (0.8): steilste Hs-daling bij s = -7 m
```

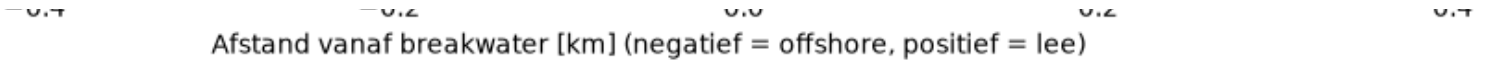

NE - Unhealthy mangroves (0.7)
Significant wave height Hs [m]
0.00
0.05
0.10
0.15
0.20
0.25
0.30
0.35
Baseline mean
Baseline median
Intervention mean
Intervention median
Breakwater (interventie steilste daling)


NE - Healthy mangroves (0.8)
Significant wave height Hs [m]
0.00
0.05
0.10
0.15
0.20
0.25
0.30
0.35
0.40




```
SW Unhealthy mangroves (0.7): steilste Hs-daling bij s = -29 m
SW Healthy mangroves (0.8): steilste Hs-daling bij s = -14 m
```

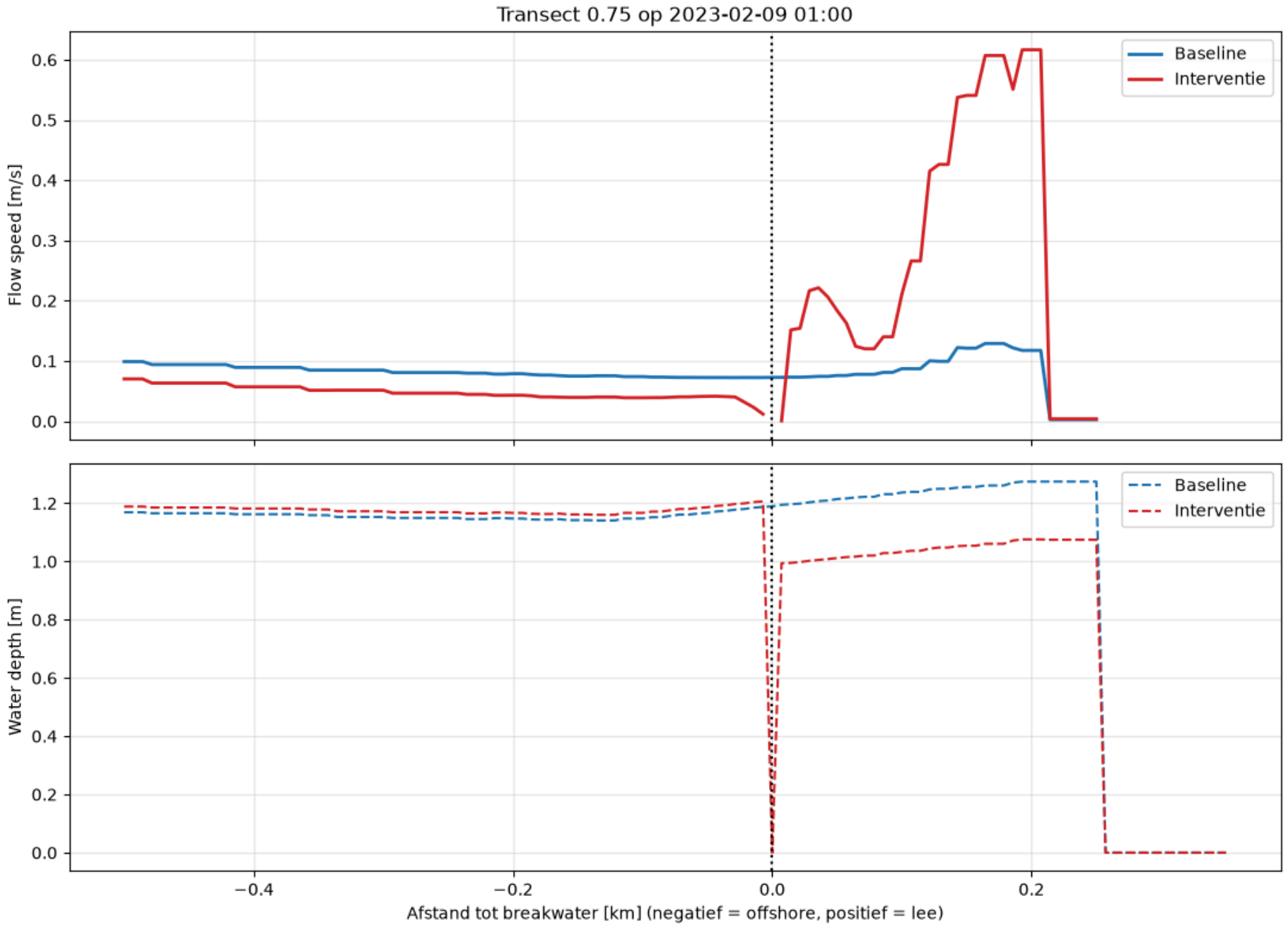

SW - Unhealthy mangroves (0.7)
Significant wave height Hs [m]
0.25
0.20
0.15
0.10
0.05
0.00
Baseline mean
Baseline median
Intervention mean
Intervention median
Breakwater (interventie steilste daling)
SW - Healthy mangroves (0.8)
Significant wave height Hs [m]
0.25
0.20
0.15
0.10
0.05
0.00
−0.4
−0.2
0.0
0.2
0.4



7.3 Snapshot of a transect

In [21]:
```
def transect_snapshot(key_i, key_b, when, fraction):
    """Snelheid en diepte langs het transect op één tijdstip (dichtstbijzijnde tijdstap per run)."""
    ds_i, ds_b = flow(key_i), flow(key_b)
    t = np.datetime64(when)
    ti = int(np.argmin(np.abs(ds_i['time'].values - t)))
    tb = int(np.argmin(np.abs(ds_b['time'].values - t)))
    pi, pb = flow_transect_profile(ds_i, fraction, ti), flow_transect_profile(ds_b, fraction, tb)
    fig, axes = plt.subplots(2, 1, figsize=(11, 8), sharex=True)
    axes[0].plot(pb['s'], pb['speed'][0], label='Baseline', color='tab:blue', linewidth=2)
    axes[0].plot(pi['s'], pi['speed'][0], label='Interventie', color='tab:red', linewidth=2)
    axes[0].set_ylabel('Flow speed [m/s]')
    axes[0].set_title(f'Transect {fraction} op {time_str(ds_i["time"].values[ti])}')
    axes[1].plot(pb['s'], pb['depth'][0], label='Baseline', color='tab:blue', linestyle='--')
    axes[1].plot(pi['s'], pi['depth'][0], label='Interventie', color='tab:red', linestyle='--')
    axes[1].set_ylabel('Water depth [m]')
    axes[1].set_xlabel('Afstand tot breakwater [km] (negatief = offshore, positief = lee)')
    for ax in axes:
        ax.axvline(0, color='black', linestyle=':')
        ax.grid(alpha=0.3)
        ax.legend()
    plt.tight_layout()
    plt.show()


transect_snapshot('ne_intervention', 'ne_baseline', '2023-02-09T01:00:00', 0.75)
```


Transect 0.75 op 2023-02-09 01:00
Flow speed [m/s]
0.0
0.1
0.2
0.3
0.4
0.5
0.6
Baseline
Interventie
Water depth [m]
0.0
0.2
0.4
0.6
0.8
1.0
1.2
Baseline
Interventie
−0.4
−0.2
0.0
0.2
Afstand tot breakwater [km] (negatief = offshore, positief = lee)

7.4 Diagnostic of flowvelocity peaks at lee side

Wetting drying check and height difference over the breakwater.

In [22]:
```
def bed_level(ds, face, s1, depth):
    for name in ('mesh2d_flowelem_bl', 'mesh2d_bedlevel'):
        if name in ds:
            var = ds[name]
            pick = dict(mesh2d_nFaces=face)
            if 'time' in var.dims:
                pick['time'] = 0
            return float(var.isel(**pick).values)
    return float(np.nanmedian(s1 - depth))                # afgeleid: waterstand - diepte


def find_lee_hotspot(ds_i, time_idx):
    """Face aan de lee-zijde (s > 0, beide transecten) met de hoogste piek-snelheid."""
    best, best_val = None, -np.inf
    for frac in TRANSECTS:
        p = flow_transect_profile(ds_i, frac, time_idx)
        lee = p['s'] > 0
        with warnings.catch_warnings():
            warnings.simplefilter('ignore', RuntimeWarning)
            peak = np.nanmax(p['speed'][:, lee], axis=0)
        if np.isfinite(peak).any() and np.nanmax(peak) > best_val:
            best_val = float(np.nanmax(peak))
            best = int(p['faces'][lee][int(np.nanargmax(peak))])
    return best


def lee_side_diagnostic(key_i, key_b, label, face=None, n_days=10, thr=VELOCITY_THRESHOLD):
    ds_i, ds_b = flow(key_i), flow(key_b)
    assert_same_mesh(ds_i, ds_b)
    c, ib, ii = common_times(ds_b, ds_i, start=SPINUP_H)
    n = 24 * n_days
    c, ib, ii = c[:n], ib[:n], ii[:n]
    if face is None:
        face = find_lee_hotspot(ds_i, ii)
    print(f'--- {label}: face {face} ---')
```

```
def series(ds, idx):
    pick = dict(time=idx, mesh2d_nFaces=face)
    s1 = ds['mesh2d_s1'].isel(**pick).values
    sp = np.hypot(ds['mesh2d_ucx'].isel(**pick).values, ds['mesh2d_ucy'].isel(**pick).values)
    dp = ds['mesh2d_waterdepth'].isel(**pick).values
    return s1, sp, dp

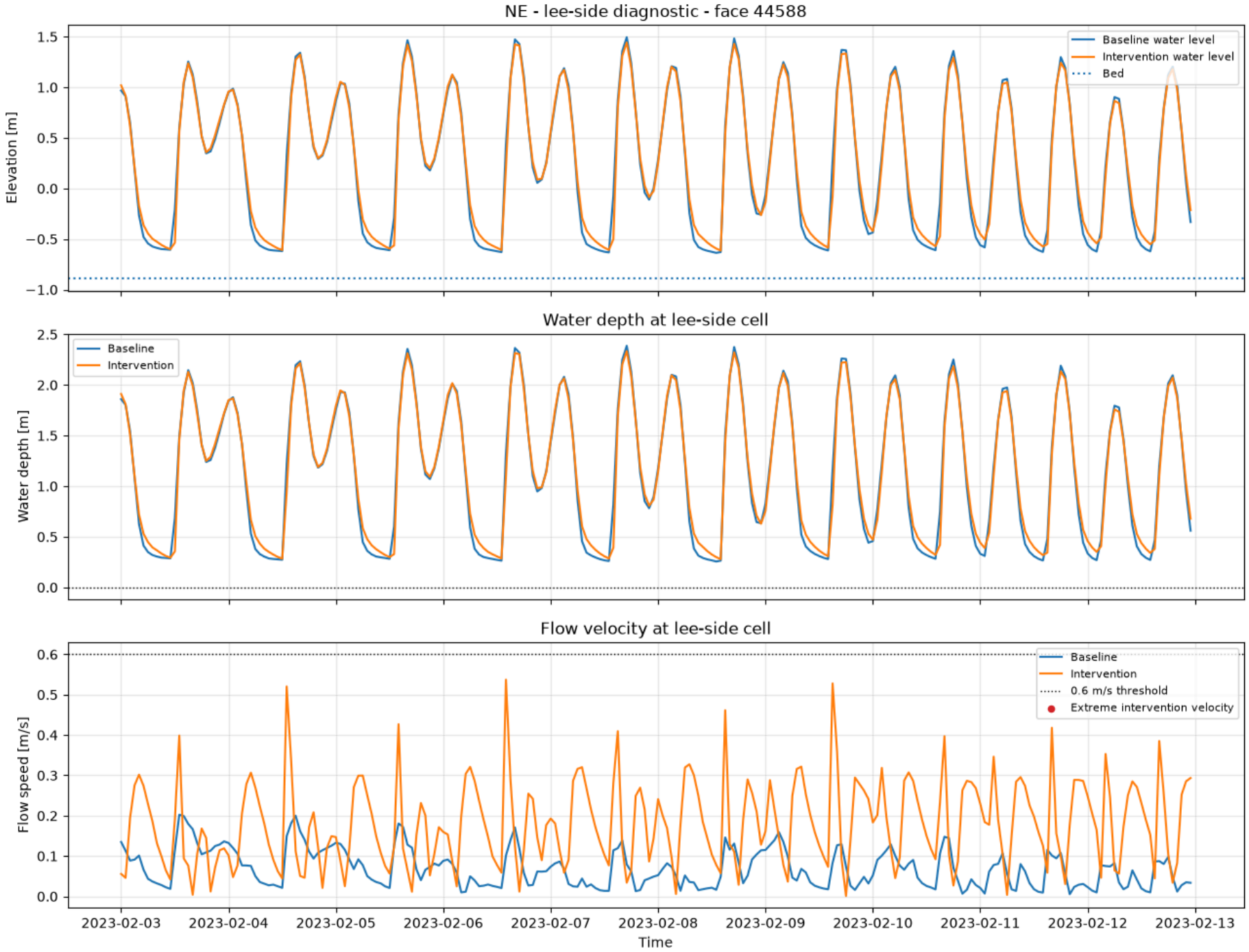
s1b, ub, db_ = series(ds_b, ib)
s1i, ui, di_ = series(ds_i, ii)
bed_b, bed_i = bed_level(ds_b, face, s1b, db_), bed_level(ds_i, face, s1i, di_)
extreme = ui > thr
rise_i = np.gradient(s1i)                                # [m per uur]; uurlijkse uitvoer
print(f'Bodem baseline {bed_b:.2f} m | interventie {bed_i:.2f} m')
print(f'Min waterdiepte in reeks: baseline {np.nanmin(db_):.2f} m, interventie {np.nanmin(di_):.2f} m')
print(f'Max snelheid: interventie {np.nanmax(ui):.2f} m/s, baseline {np.nanmax(ub):.2f} m/s')
print(f'Aantal uren > {thr} m/s: {int(extreme.sum())}')
if extreme.any():
    print(f'Diepte tijdens extremen: {np.nanmin(di_[extreme]):.2f} - {np.nanmax(di_[extreme]):.2f} m')
    print(f'Waterstand-stijging tijdens extremen: mediaan {np.nanmedian(rise_i[extreme]):+.2f} m/uur '
          f'(alle uren: {np.nanmedian(rise_i):+.2f} m/uur)')

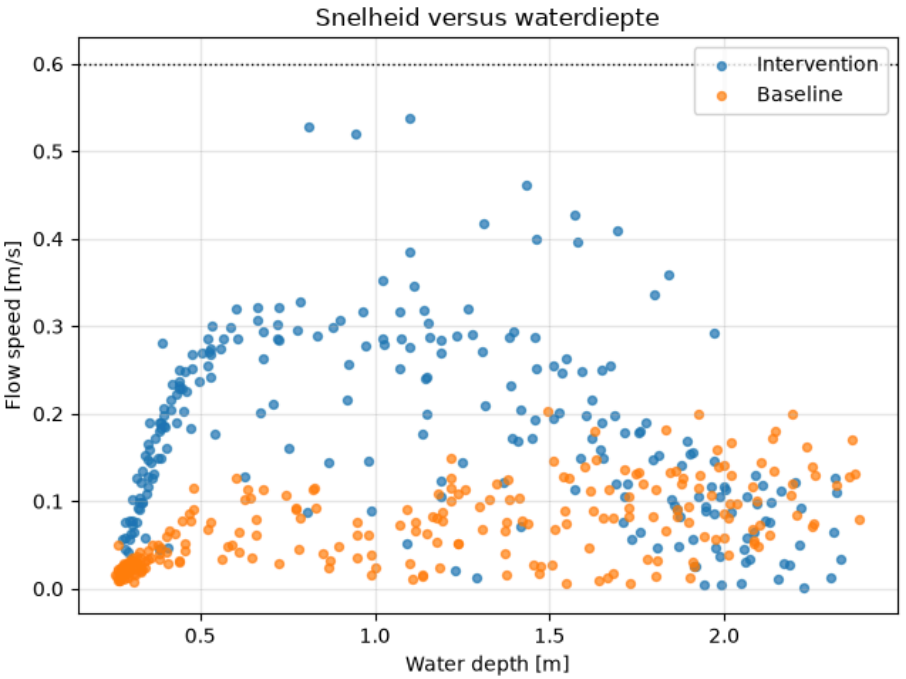
fig, ax = plt.subplots(3, 1, figsize=(13, 10), sharex=True)
ax[0].plot(c, s1b, label='Baseline water level')
ax[0].plot(c, s1i, label='Intervention water level')
ax[0].axhline(bed_b, linestyle=':', color='tab:blue', label='Bed')
ax[0].set(ylabel='Elevation [m]', title=f'{label} - lee-side diagnostic - face {face}')
ax[1].plot(c, db_, label='Baseline')
ax[1].plot(c, di_, label='Intervention')
ax[1].axhline(0, color='black', linestyle=':', linewidth=1)
ax[1].set(ylabel='Water depth [m]', title='Water depth at lee-side cell')
ax[2].plot(c, ub, label='Baseline')
ax[2].plot(c, ui, label='Intervention')
ax[2].axhline(thr, color='black', linestyle=':', linewidth=1, label=f'{thr} m/s threshold')
ax[2].scatter(c[extreme], ui[extreme], s=20, zorder=5, color='tab:red', label='Extreme intervention velocity')
ax[2].set(ylabel='Flow speed [m/s]', xlabel='Time', title='Flow velocity at lee-side cell')
for a in ax:
    a.legend(fontsize=8)
    a.grid(alpha=0.3)
plt.tight_layout()
plt.show()
```

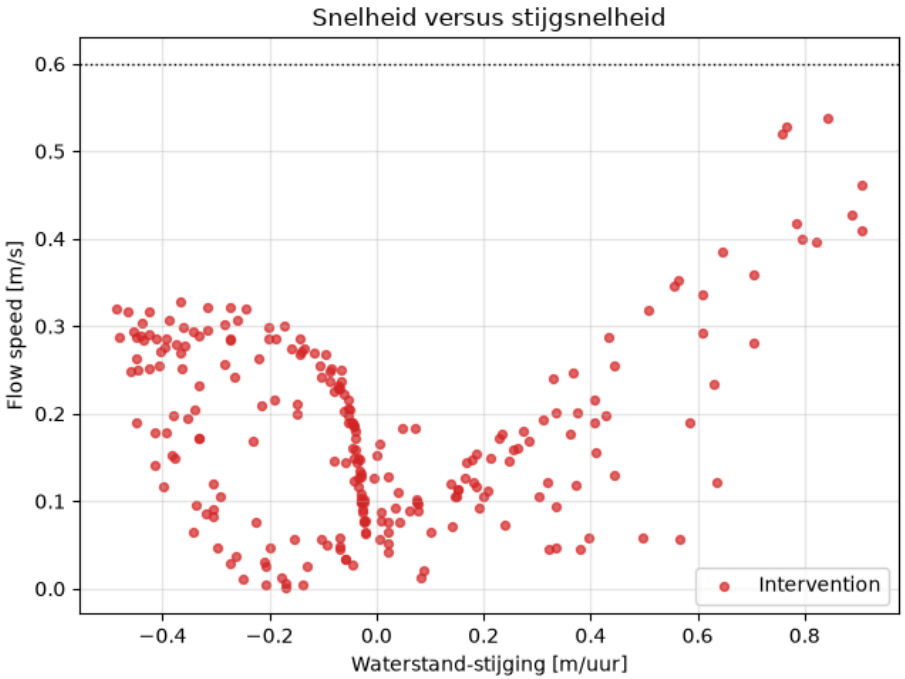
```
    fig, axs = plt.subplots(1, 2, figsize=(13, 5))
    axs[0].scatter(di_, ui, s=18, alpha=0.7, label='Intervention')
    axs[0].scatter(db_, ub, s=18, alpha=0.7, label='Baseline')
    axs[0].set(xlabel='Water depth [m]', ylabel='Flow speed [m/s]', title='Snelheid versus waterdiepte')
    axs[1].scatter(rise_i, ui, s=18, alpha=0.7, color='tab:red', label='Intervention')
    axs[1].set(xlabel='Waterstand-stijging [m/uur]', ylabel='Flow speed [m/s]', title='Snelheid versus stijgsnelheid')
    for a in axs:
        a.axhline(thr, color='black', linestyle=':', linewidth=1)
        a.grid(alpha=0.3)
        a.legend()
    plt.tight_layout()
    plt.show()


lee_side_diagnostic('ne_intervention', 'ne_baseline', 'NE')
lee_side_diagnostic('sw_intervention', 'sw_baseline', 'SW')
```

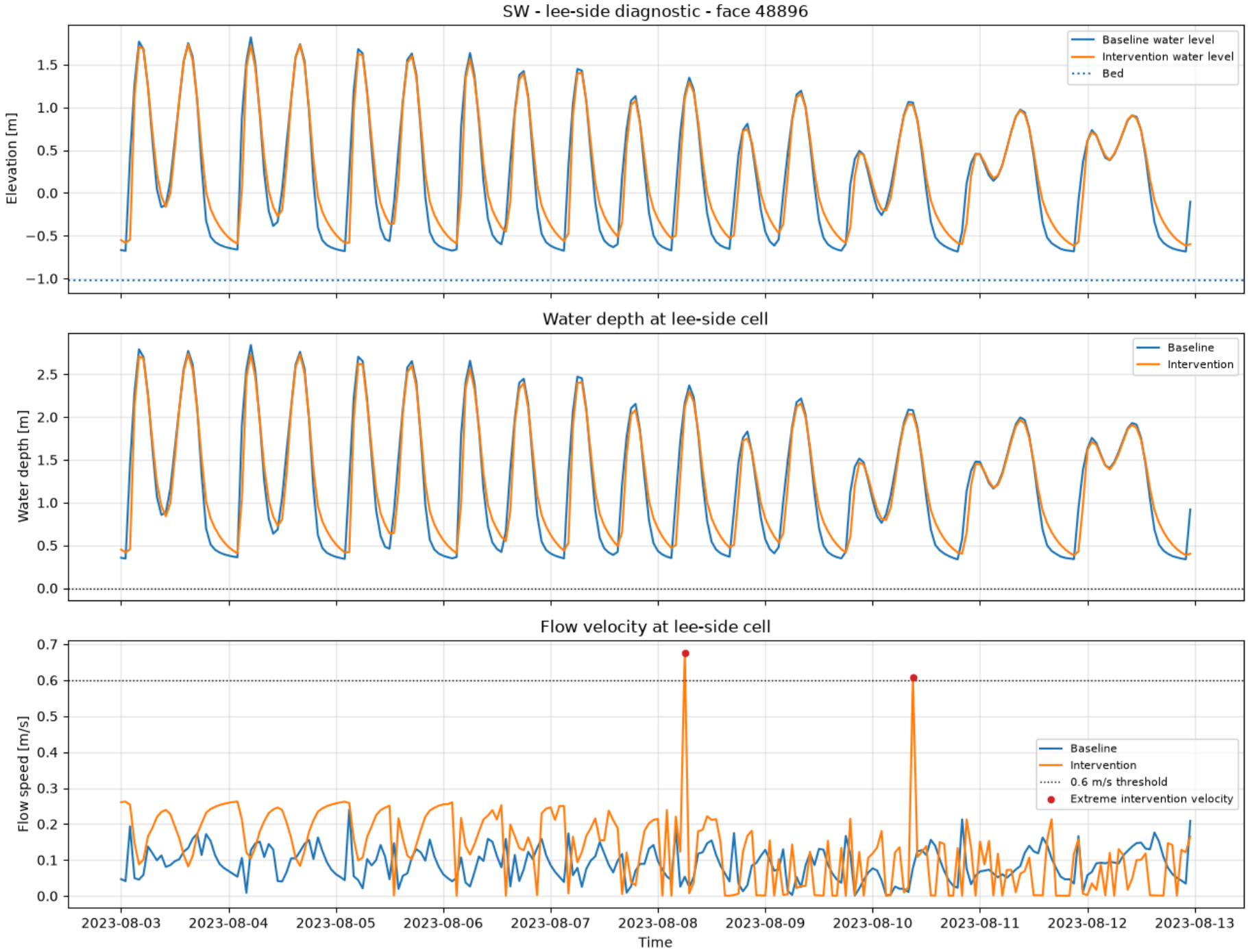
```
--- NE: face 44588 ---
Bodem baseline -0.89 m | interventie -0.89 m
Min waterdiepte in reeks: baseline 0.26 m, interventie 0.28 m
Max snelheid: interventie 0.54 m/s, baseline 0.20 m/s
Aantal uren > 0.6 m/s: 0
```

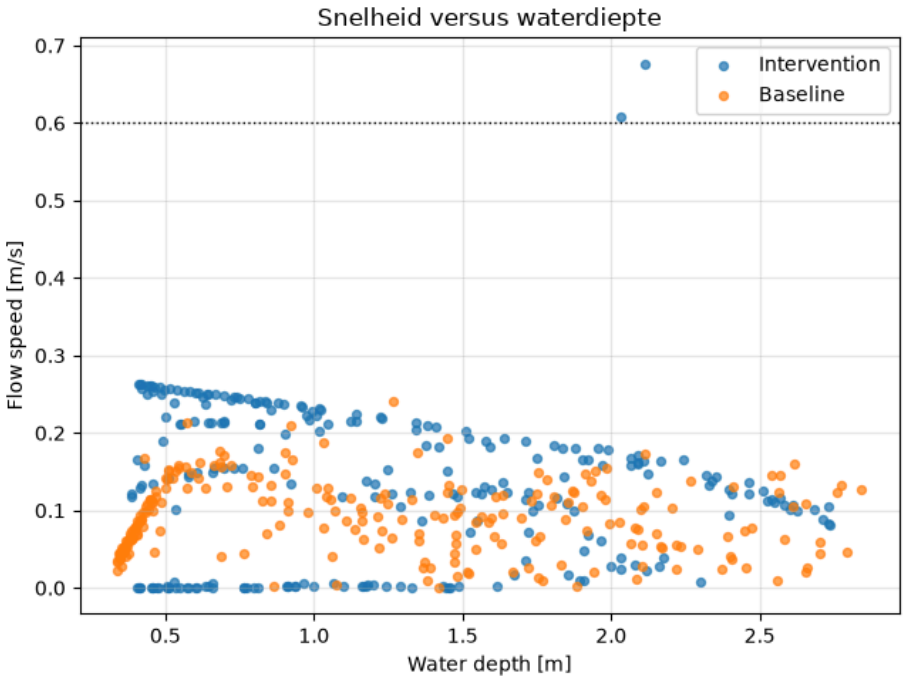

NE - lee-side diagnostic - face 44588
Elevation [m]
Baseline water level
Intervention water level
Bed
Water depth at lee-side cell
Water depth [m]
Baseline
Intervention
Flow velocity at lee-side cell
Flow speed [m/s]
Baseline
Intervention
0.6 m/s threshold
Extreme intervention velocity
2023-02-03
2023-02-04
2023-02-05
2023-02-06
2023-02-07
2023-02-08
2023-02-09
2023-02-10
2023-02-11
2023-02-12
2023-02-13
Time

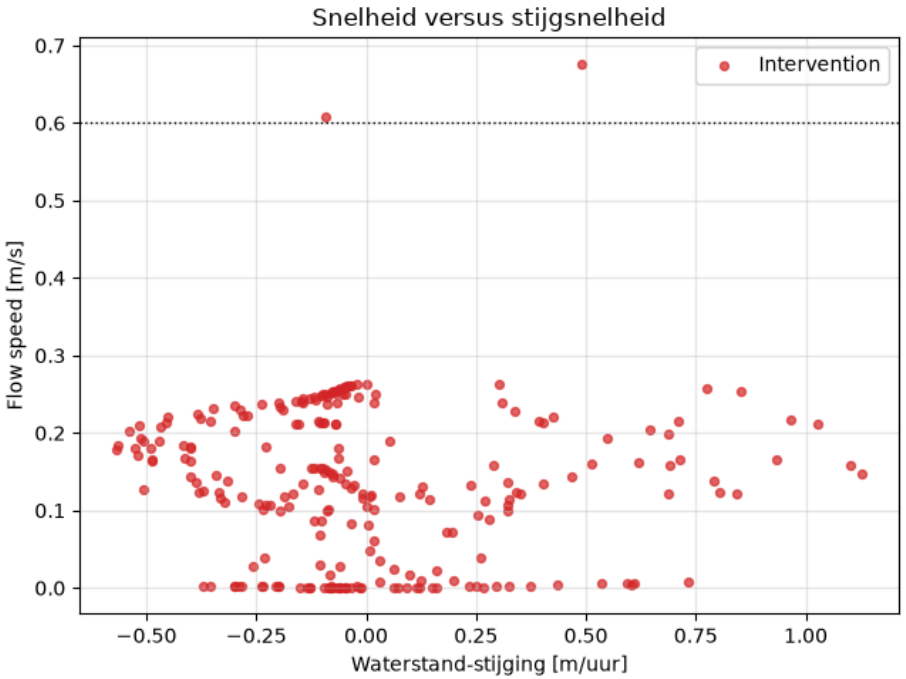





```
--- SW: face 48896 ---
Bodem baseline -1.02 m | interventie -1.00 m
Min waterdiepte in reeks: baseline 0.34 m, interventie 0.38 m
Max snelheid: interventie 0.68 m/s, baseline 0.24 m/s
Aantal uren > 0.6 m/s: 2
Diepte tijdens extremen: 2.03 - 2.11 m
Waterstand-stijging tijdens extremen: mediaan +0.20 m/uur (alle uren: -0.07 m/uur)
```


SW - lee-side diagnostic - face 48896
Elevation [m]
Baseline water level
Intervention water level
Bed
Water depth at lee-side cell
Water depth [m]
Baseline
Intervention
Flow velocity at lee-side cell
Flow speed [m/s]
Baseline
Intervention
0.6 m/s threshold
Extreme intervention velocity
2023-08-03
2023-08-04
2023-08-05
2023-08-06
2023-08-07
2023-08-08
2023-08-09
2023-08-10
2023-08-11
2023-08-12
2023-08-13
Time


Snelheid versus waterdiepte
Intervention
Baseline
Flow speed [m/s]
Water depth [m]
0.0
0.1
0.2
0.3
0.4
0.5
0.6
0.7
0.5
1.0
1.5
2.0
2.5


Snelheid versus stijgsnelheid
Intervention
Flow speed [m/s]
Waterstand-stijging [m/uur]
0.0
0.1
0.2
0.3
0.4
0.5
0.6
0.7
−0.50
−0.25
0.00
0.25
0.50
0.75
1.00